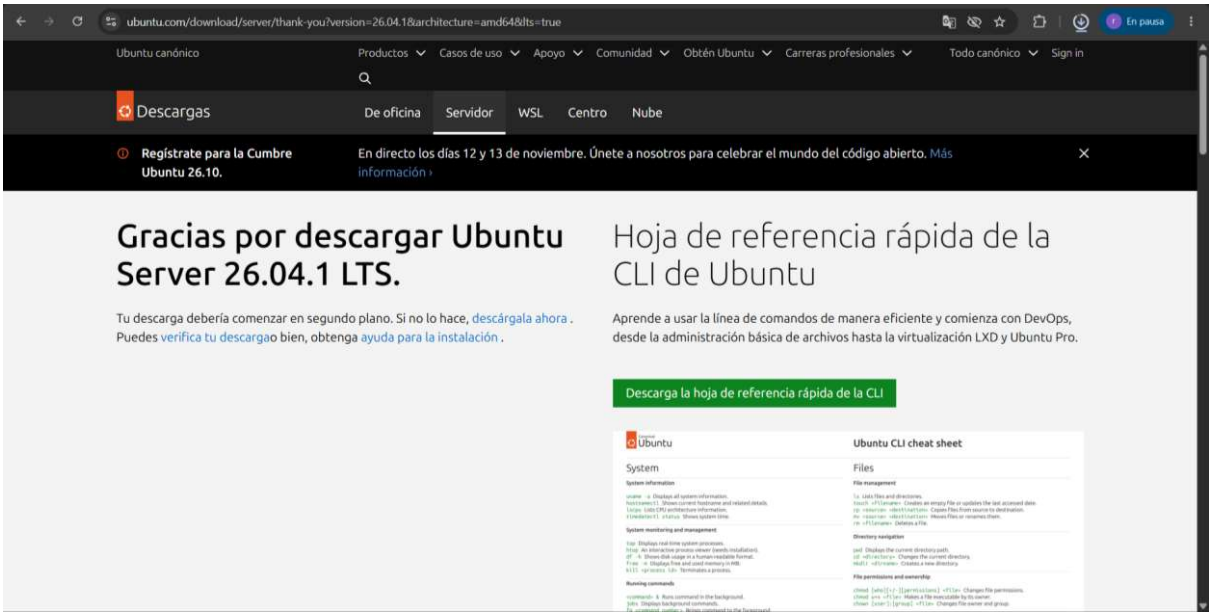
Parcial 2 Sistemas Operativos

Por: Richard Delgado

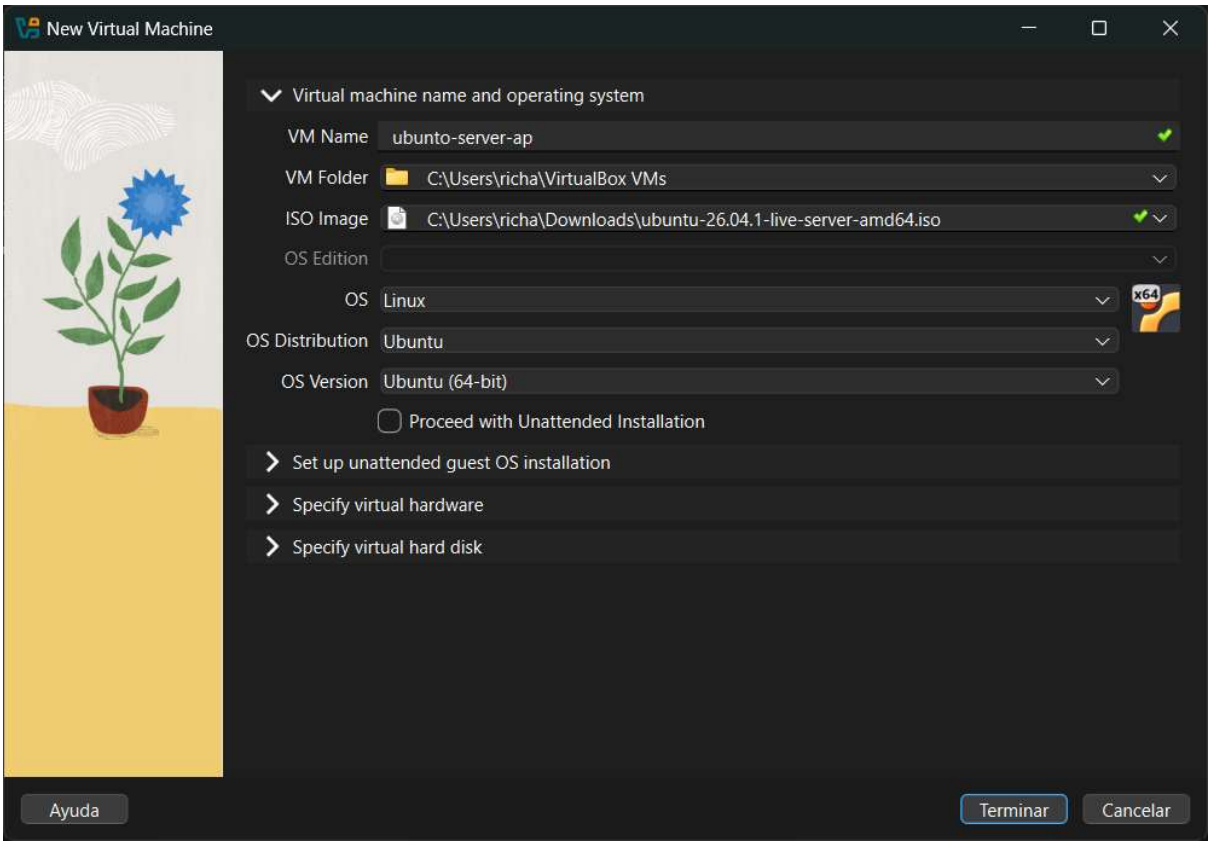




1,2) Descarga de la imagen ISO de Ubuntu Server

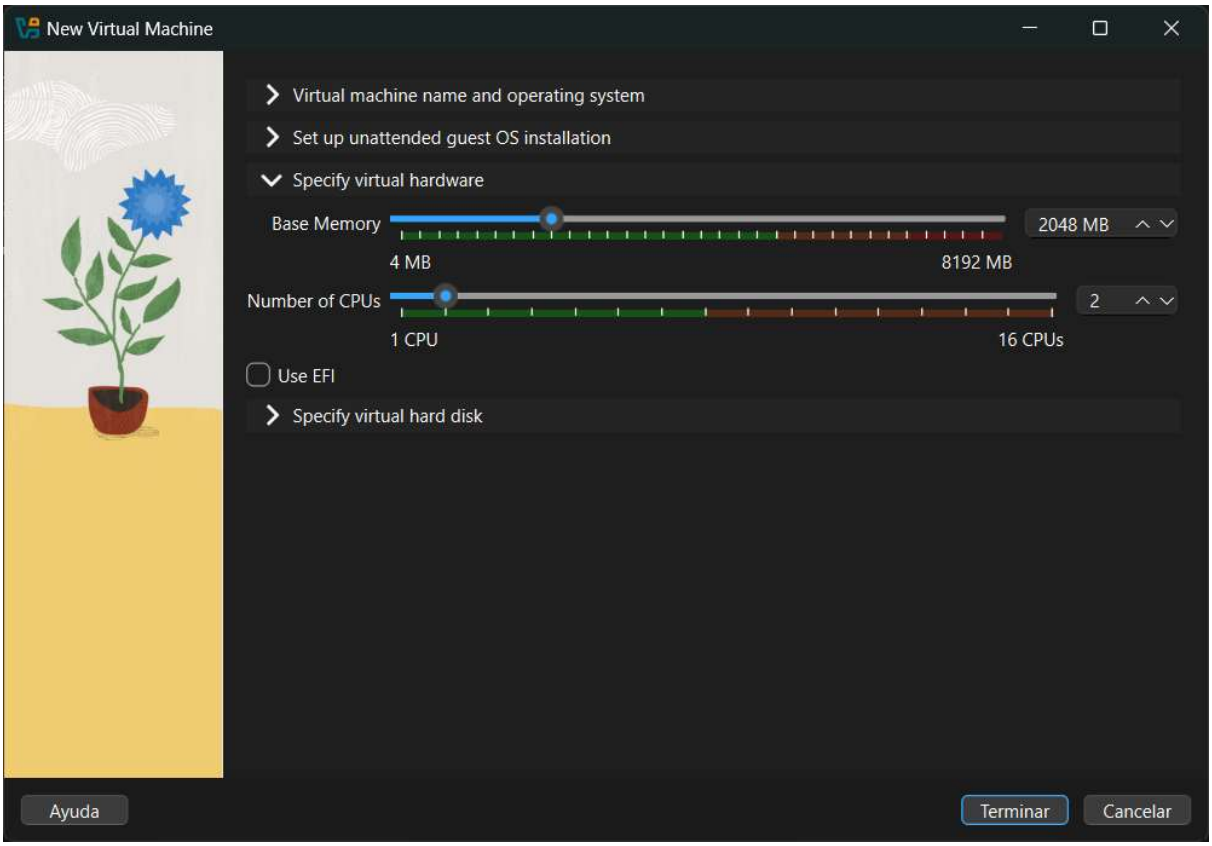
Para el equipo servidor se descargó la imagen ISO de Ubuntu Server 26.04.1 LTS desde el sitio oficial de Canonical (ubuntu.com/download/server). Se eligió la versión Server, sin entorno gráfico, porque consume menos recursos y es la que pide el taller para el servidor.

La descarga terminó correctamente. El archivo ubuntu-26.04.1-live-server-amd64.iso pesa 2,7 GB y se usará como medio de instalación en la máquina virtual del servidor.



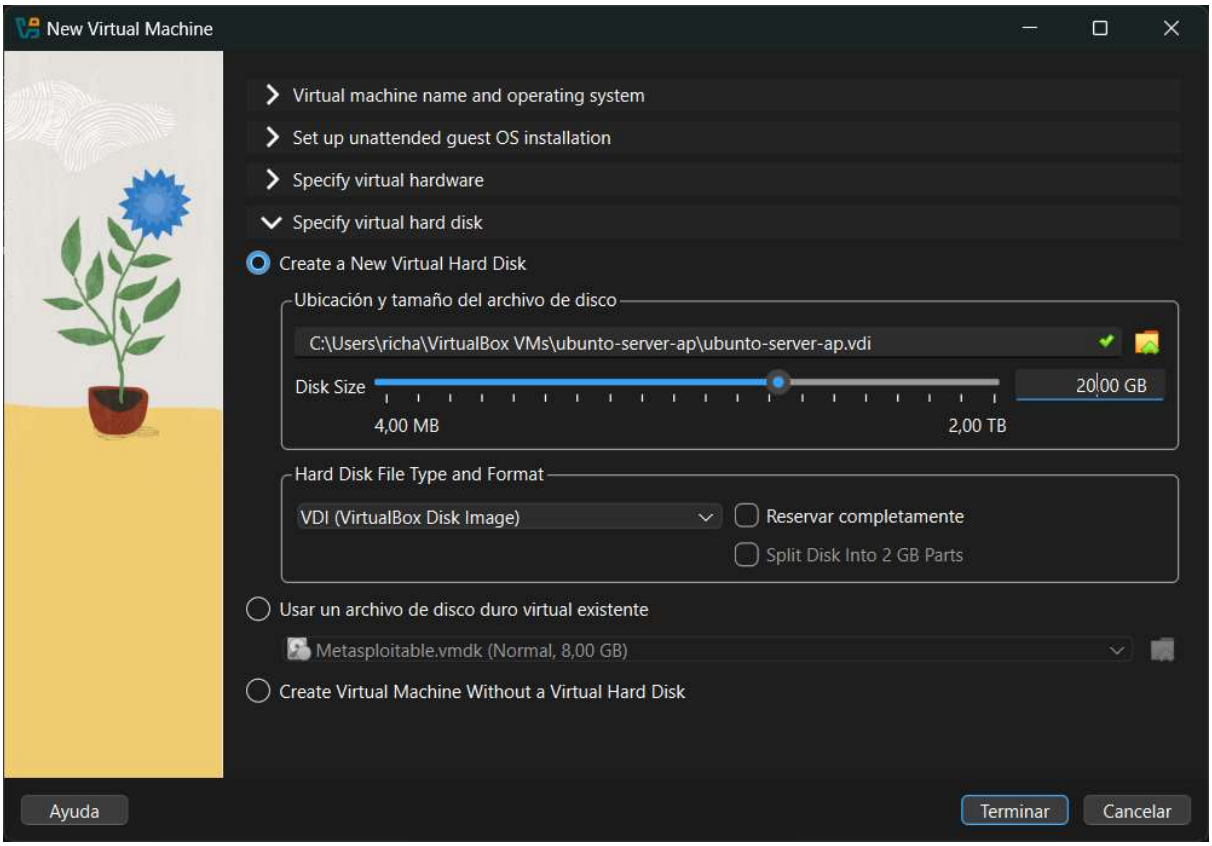
3) Creación de la máquina virtual

Se creó una nueva máquina virtual en VirtualBox con el nombre ubunto-server-ap. Se seleccionó como imagen de instalación el ISO descargado, y el sistema se configuró como Linux, distribución Ubuntu (64-bit). Se dejó desmarcada la opción de instalación desatendida para realizar la instalación de forma manual y poder configurar el usuario, el nombre del equipo y OpenSSH



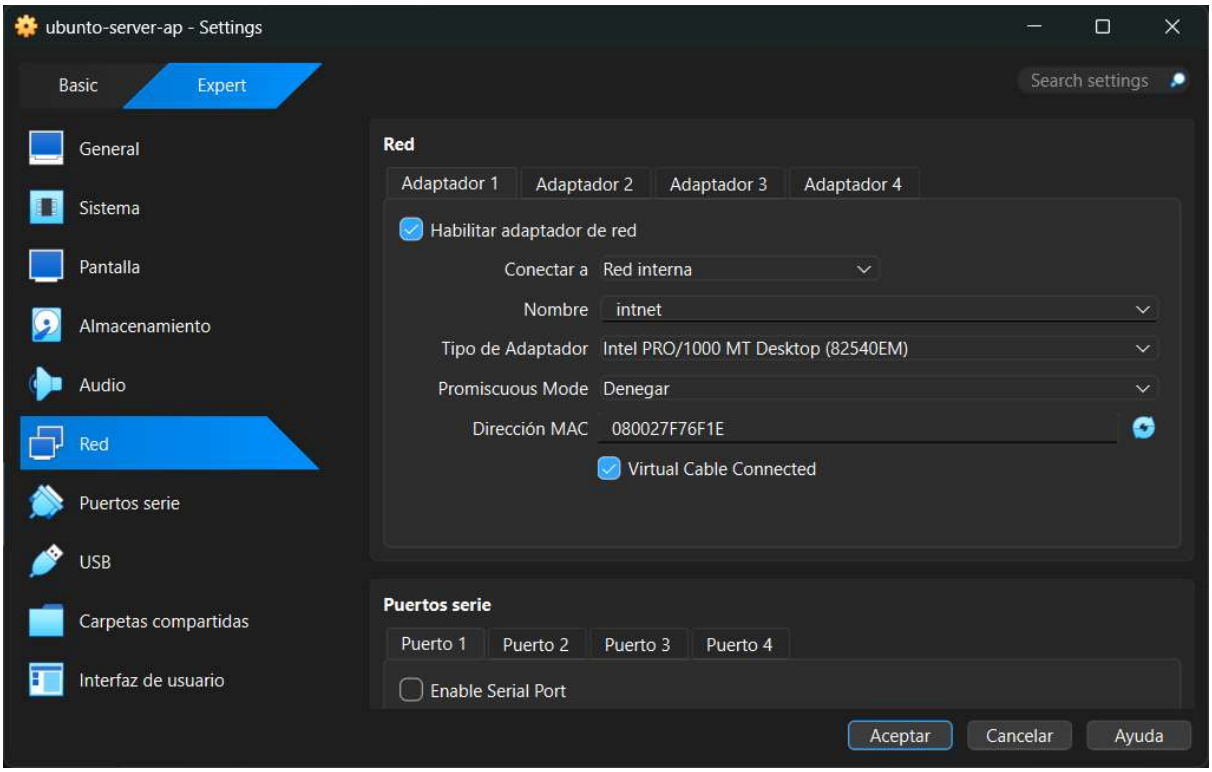
4) Asignación de memoria RAM y procesadores

Se asignaron al servidor 2048 MB (2 GB) de memoria RAM y 2 procesadores. Son recursos suficientes para ejecutar Ubuntu Server y un modelo de IA liviano, y a la vez permiten que el equipo anfitrión, de 8 GB de RAM, siga funcionando con normalidad.



5) Creación del disco virtual

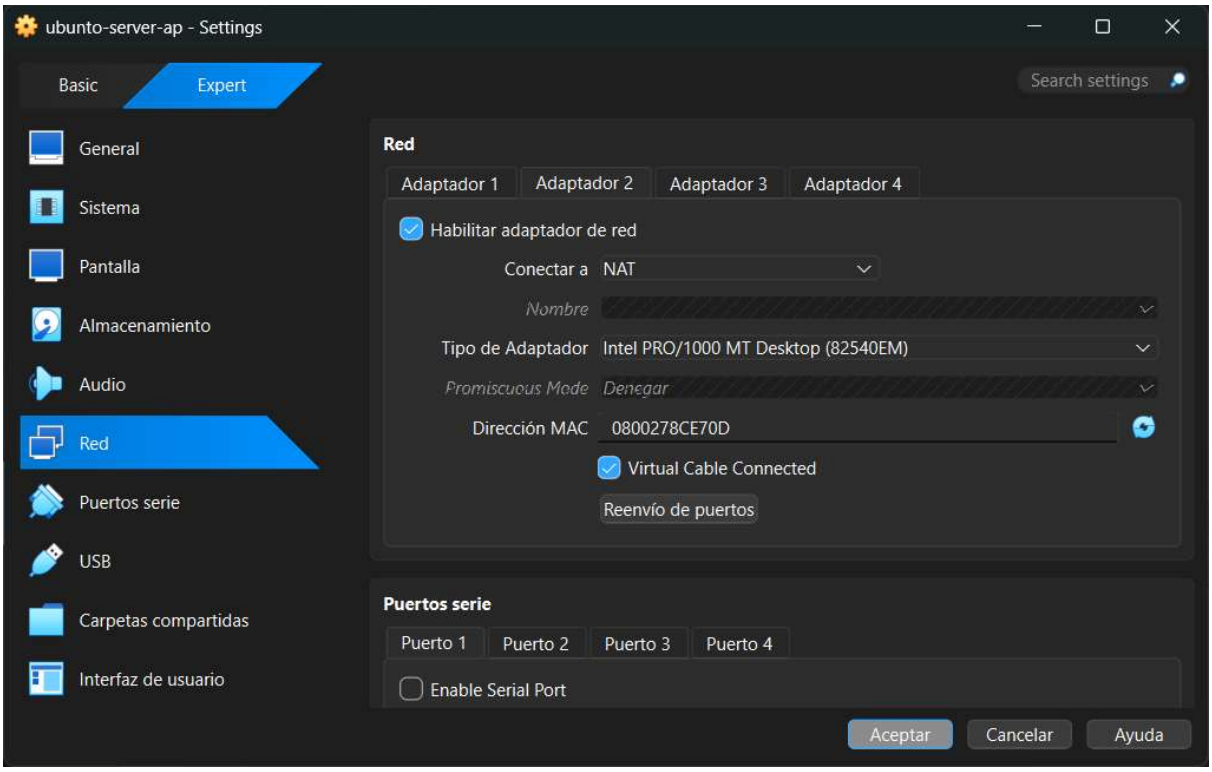
Se creó un disco virtual nuevo de 20 GB en formato VDI (VirtualBox Disk Image), almacenado en la carpeta de máquinas virtuales de VirtualBox. Este espacio es suficiente para el sistema operativo, las carpetas de almacenamiento y el modelo de IA, que pesa 397 MB.



6) Configuración del Adaptador 1 (red interna)

Adaptador 1 (Red interna, intnet): conecta el servidor con el cliente. Es la red privada del taller, donde viajan el ping, SSH y SCP. No tiene internet.

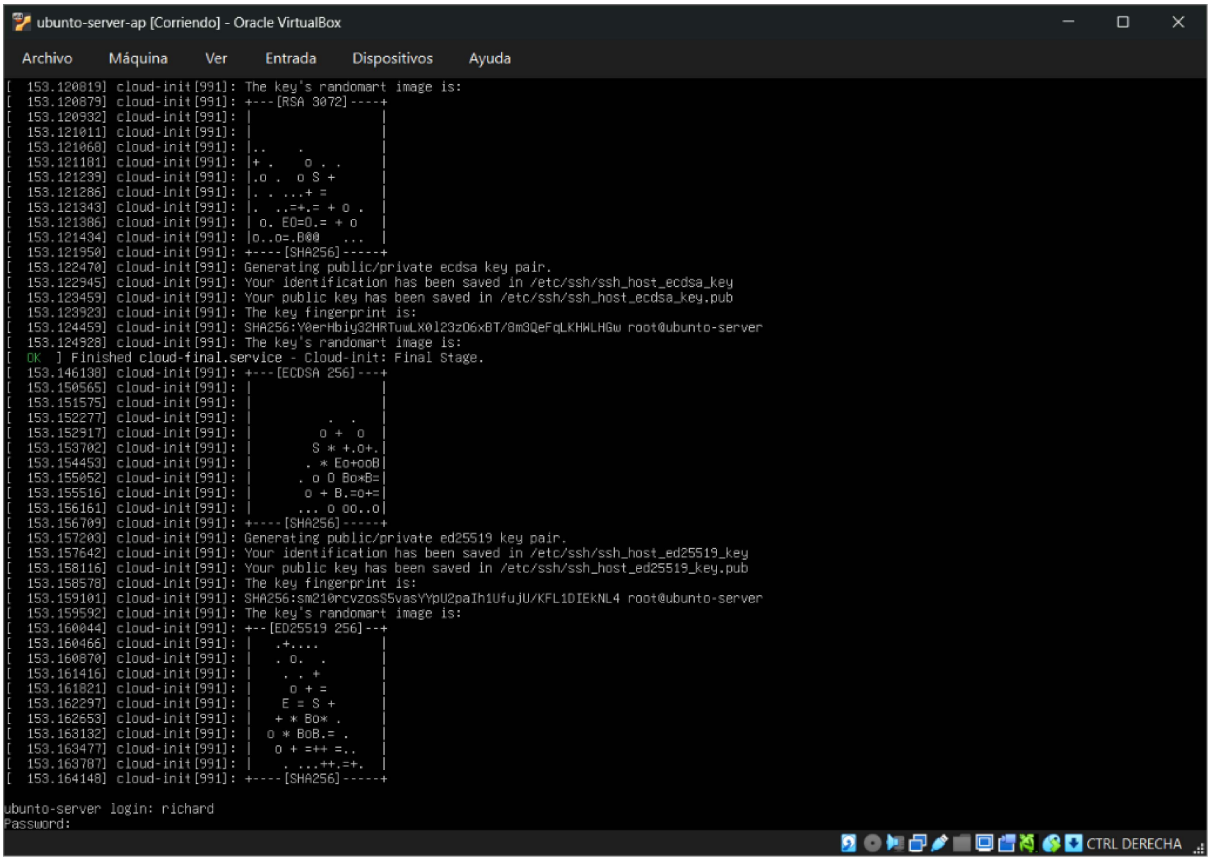
Adaptador 2 (NAT): da salida a internet al servidor. Se necesita para instalar paquetes y descargar Ollama y el modelo.



7) Configuración del Adaptador 2 (NAT)

El Adaptador 2 quedó habilitado y configurado en modo NAT, con adaptador Intel PRO/1000 MT Desktop. NAT significa “traducción de direcciones de red” (Network Address Translation) o en español traductosrde direcciones de red. Es un sistema que permite que un equipo con una dirección privada, como la máquina virtual, salga a internet usando la dirección del equipo anfitrión. Funciona como un intermediario: el servidor puede conectarse a internet, pero desde afuera nadie puede conectarse a él.

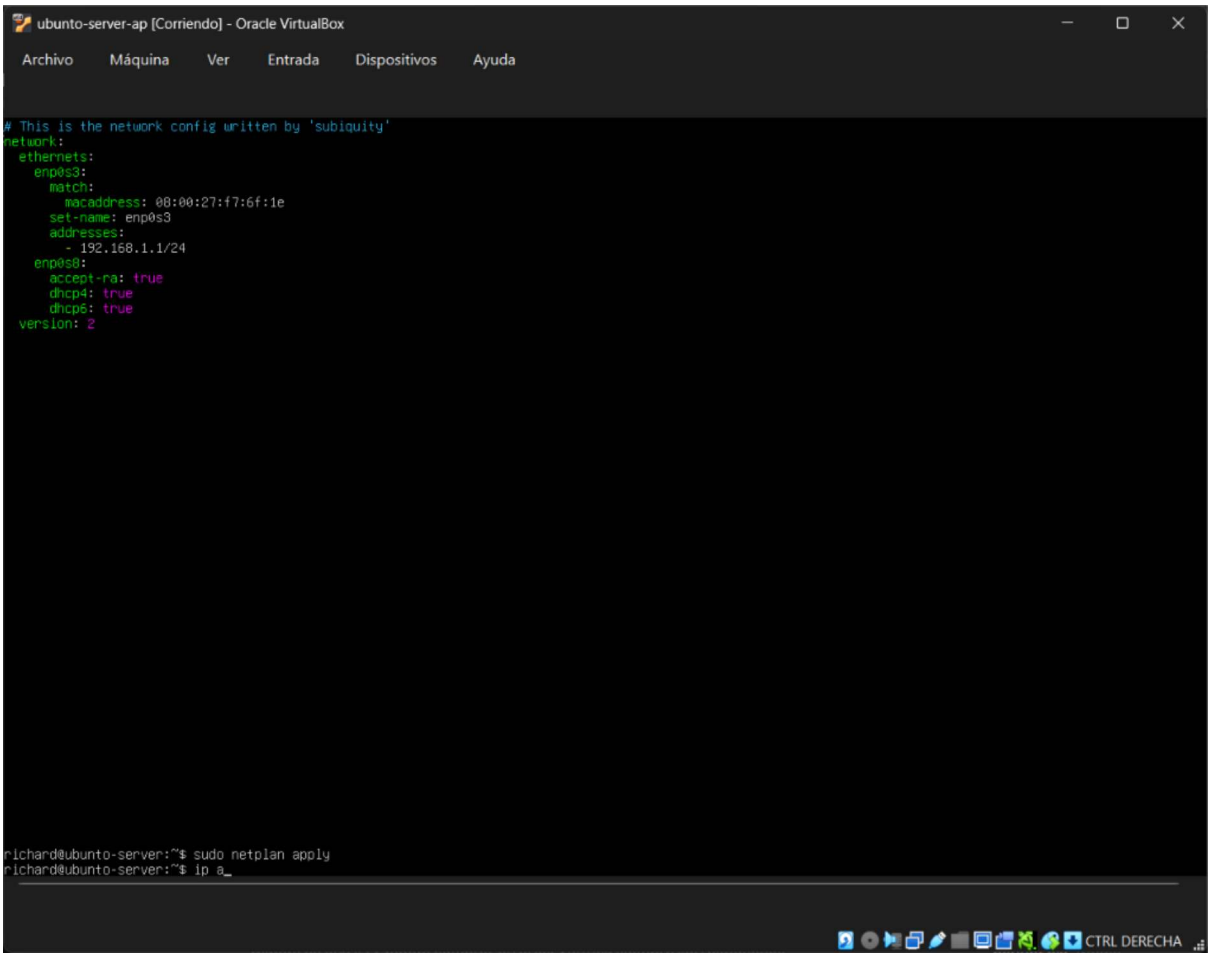
Es decir; que la NAT actúa como un filtro entre la red y la máquina virtual, se puede entender haciendo analogía de un edificio en el que llega una carta con la dirección de dicho edificio, pero la NAT; en este caso encarna el rol del portero que solo sabe cuál es el número del apartamento impidiendo que algún agente externo acceda o se conecte directamente al apartamento.



8) Instalación de Ubuntu Server

Se instaló Ubuntu Server en la máquina virtual siguiendo el asistente de instalación, que hace una serie de preguntas antes de copiar el sistema al disco, preguntas para esa instancia del laboratorio bastante tediosas en lo personal...

Primero se eligieron el idioma (español), la distribución del teclado (Spanish) y el tipo de instalación (Ubuntu Server). Luego el asistente mostró la configuración de red, donde detectó las dos interfaces de la máquina virtual, y pidió datos opcionales como el proxy y el servidor espejo de descargas, que se dejaron por defecto. Después se configuró el almacenamiento, usando el disco virtual de 20 GB completo, y se creó el perfil del servidor: nombre del equipo ubuntu-server, usuario richard y su contraseña. También se activó la opción Install OpenSSH server, necesaria para el acceso remoto. Al terminar la instalación se reinició la máquina; el sistema ejecutó sus procesos de arranque inicial (cloud-init), que generan las claves SSH del servidor, y quedó en la pantalla de inicio de sesión, donde se ingresó con el usuario richard, osea; el del presente redactor; etc...



9) Configuración de la dirección IP fija con Netplan

Se le coloca al servidor un número de identificación fijo en la red del taller, para que el cliente siempre lo encuentre.

Cabe acotar comandos en adelante:

- **Netplan** es el archivo donde Ubuntu guarda la configuración de red.
- **enp0s3** es la conexión del servidor con el cliente (la red interna). Le asignamos la dirección 192.168.1.1, y esa no cambia nunca.
- **enp0s8** es la conexión a internet (NAT). Con dhcp4: true el router virtual le da su dirección automáticamente, y por eso no la fijamos.
- **sudo netplan apply** es el comando que activa los cambios.

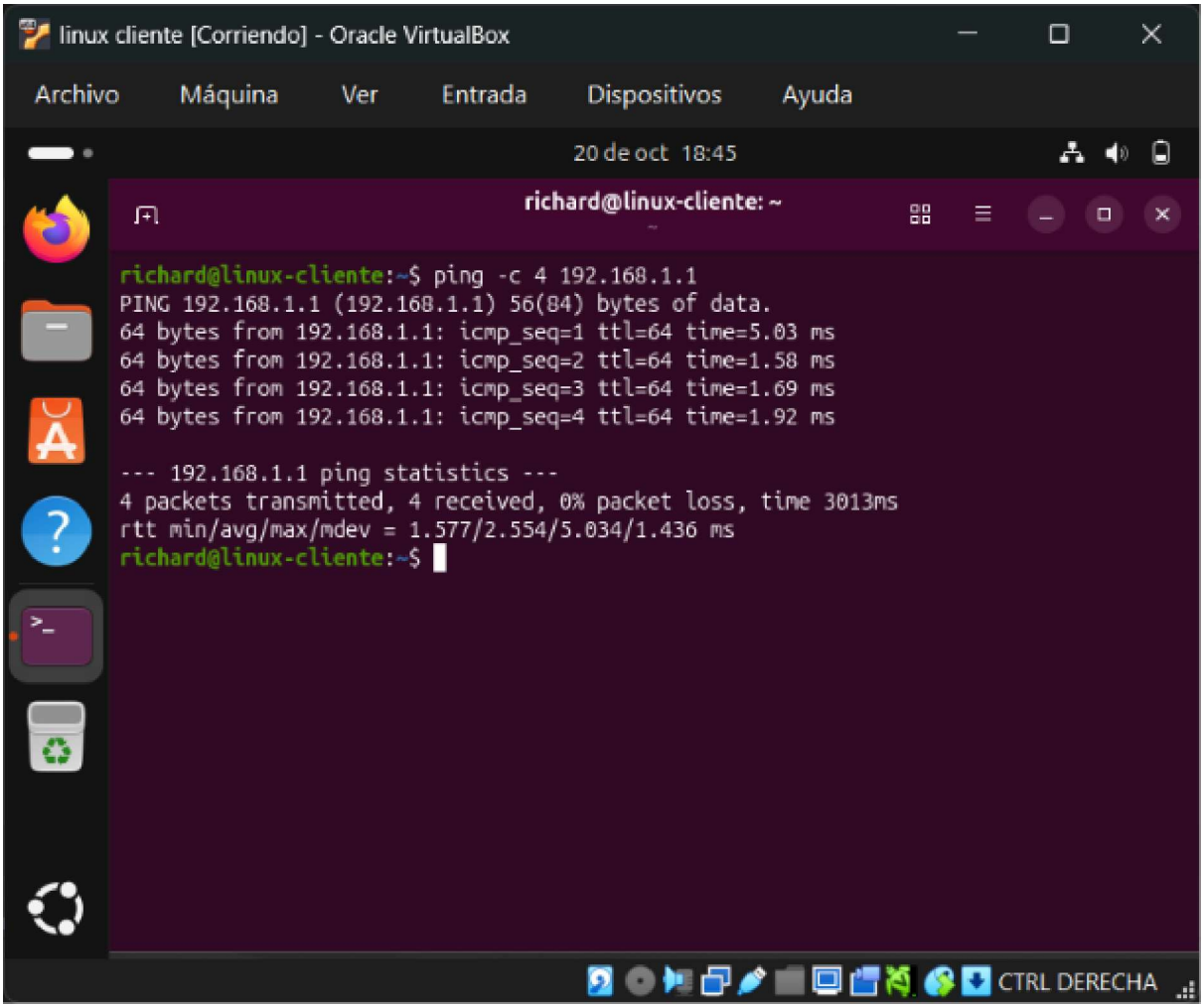
Si no fijáramos la dirección del servidor, podría cambiar cada vez que se enciende, y el cliente no sabría cómo encontrarlo para el ping, el SSH y el scp.

Es como la dirección de una casa: la de la red interna es fija para que te puedan visitar siempre, y la de internet se asigna sola.

sudo netplan apply se lee por partes:

+ **sudo**: ejecuta lo que sigue con permisos de administrador.
+ **netplan**: la herramienta que maneja la configuración de red.
+ **apply**: “aplicar”, es decir, activar los cambios guardados.

En conjunto significa: *“como administrador, aplica la configuración de red de netplan”*.



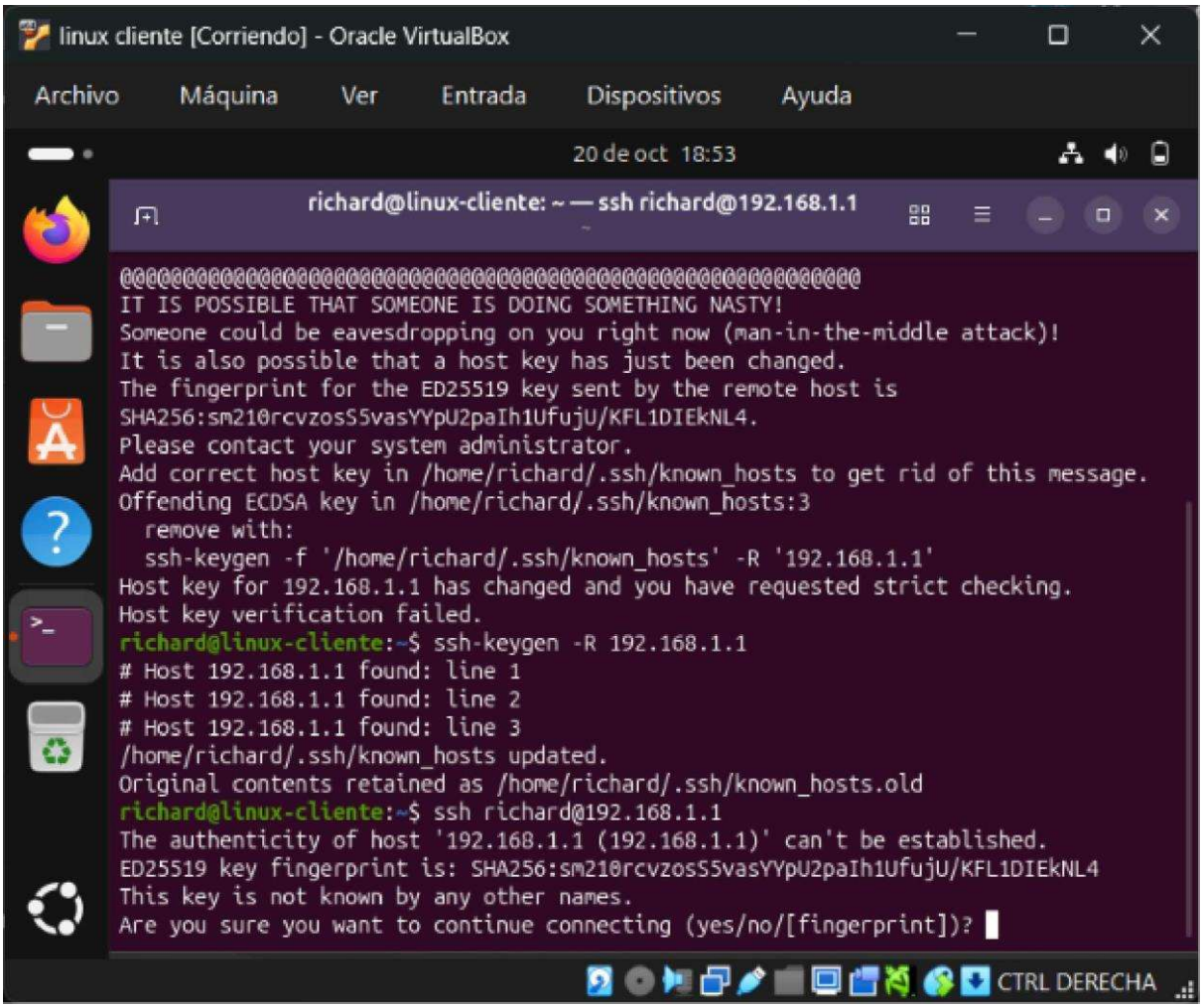
10) Verificación de conectividad con ping

Para verificar que el cliente y el servidor se ven entre sí, se realizan dos pruebas, una tras otra:

Comandos:

- **ping:** envía paquetes de prueba al servidor para comprobar si responde. Si responde, la red entre ambos equipos funciona.
- **ssh:** permite al cliente entrar al servidor de forma remota y ejecutar comandos allá, sin usar la pantalla del servidor.

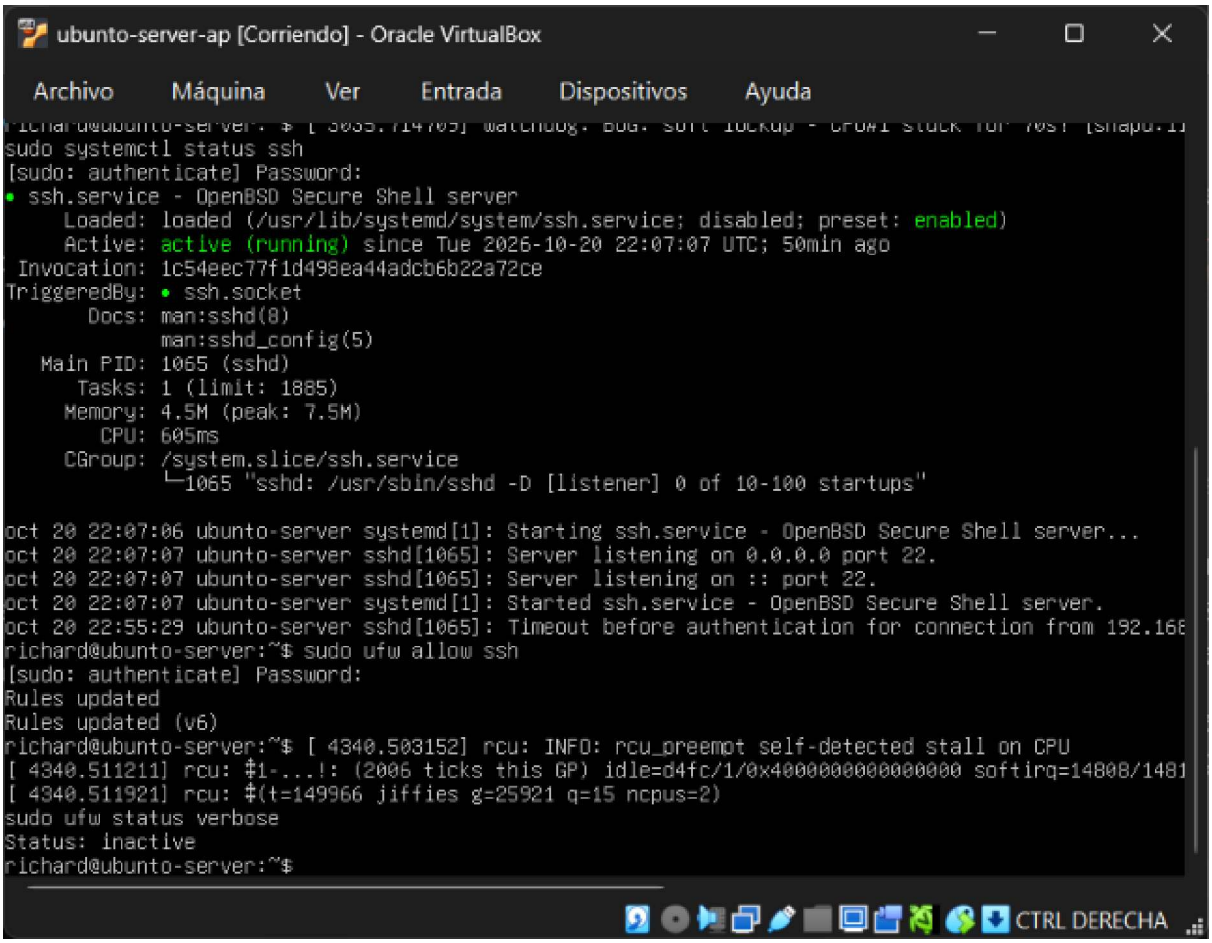
Primero se encendió la máquina cliente y, desde su terminal, se ejecutó ping -c 4 192.168.1.1, que envía 4 paquetes de prueba a la dirección del servidor (-c 4 indica la cantidad de paquetes). Se recibieron los 4 paquetes, con 0% de pérdida y tiempos de respuesta de entre 1,5 y 5 milisegundos, lo que confirma que la red interna entre el cliente y el servidor funciona correctamente. Luego se probó la conexión remota con ssh richard@192.168.1.1, como se muestra en las siguientes capturas.



11) Conexión SSH y aviso de cambio de llave del servidor

Al intentar la conexión con ssh richard@192.168.1.1, el cliente mostró una advertencia de seguridad indicando que la llave del servidor había cambiado. Esto ocurrió porque el cliente tenía guardada la llave de un servidor anterior que usó la misma dirección IP. Para resolverlo se eliminó el registro antiguo con ssh-keygen -R 192.168.1.1, que borra esa dirección del archivo known_hosts. Al repetir la conexión apareció la pregunta normal de la primera vez: *"Are you sure you want to continue connecting (yes/no/[fingerprint])?"*, que consulta si se confía en el servidor. La huella

que muestra (SHA256:sm210rcvzosS5vasYYpU2palh1UfujU/KFL1DIEkNL4) coincide con la que el servidor mostró al arrancar, por lo que es el equipo correcto. Se escribió yes completo para aceptar y continuar.



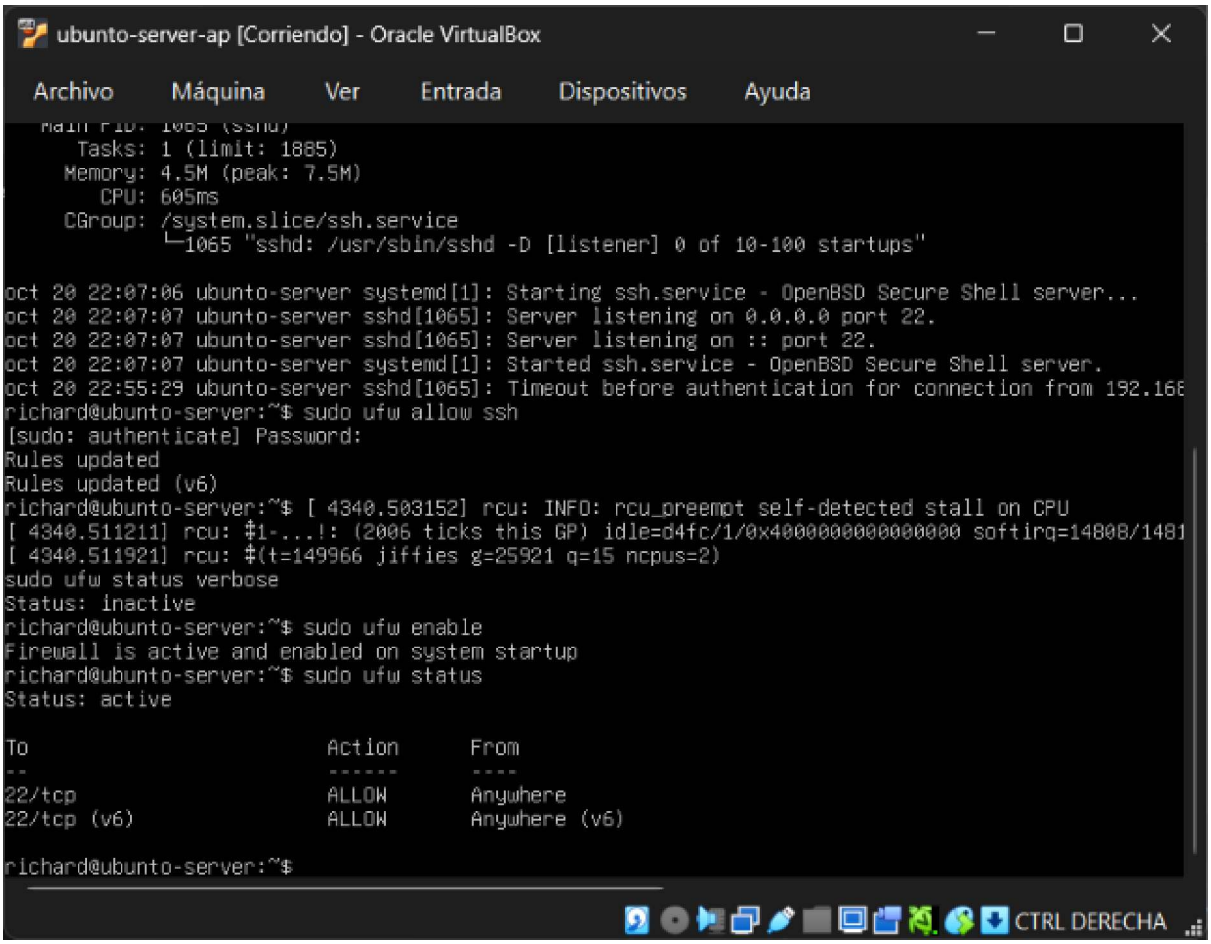
12) Verificación del servicio SSH y creación de la regla en UFW

previo a configurar el firewall se verificó que el servicio OpenSSH estuviera en funcionamiento con sudo systemctl status ssh, que mostró el estado active (running) y el servidor escuchando en el puerto 22. Luego se creó la regla que permite el tráfico SSH con sudo ufw allow ssh, que respondió Rules updated y Rules updated (v6), una por IPv4 y otra por IPv6. Con sudo ufw status verbose se comprobó que el firewall seguía inactivo (Status: inactive), por lo que la regla quedó guardada pero aún no se aplicaba. La regla se creó antes de activar el firewall para no perder el acceso remoto al servidor.

Comandos:

- **sudo systemctl status ssh:** “como administrador, muestra el estado del servicio ssh”. systemctl es la herramienta que controla los servicios del sistema, y status pide su estado.

- **sudo ufw allow ssh:** “como administrador, en el firewall, permite el servicio ssh”. ufw es el firewall, y allow significa “permitir”.
- **sudo ufw status verbose:** “como administrador, muestra el estado del firewall con detalle”. status pide el estado y verbose agrega más información.

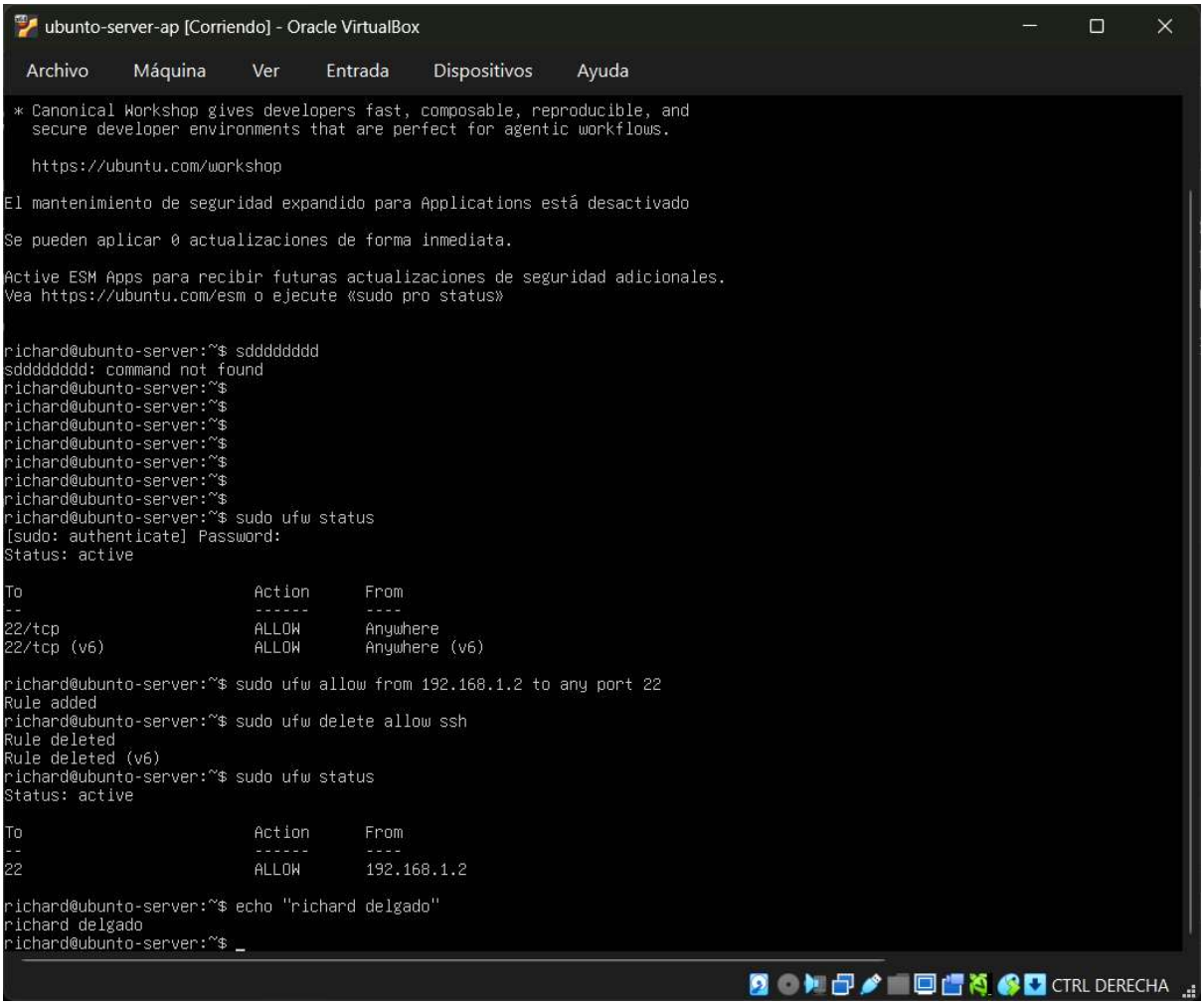


13) Activación del firewall

Con el servicio SSH activo y la regla creada, se activó el firewall con sudo ufw enable, que respondió Firewall is active and enabled on system startup, es decir, que quedó activo y se iniciará automáticamente con el sistema. Luego, con sudo ufw status, se comprobó el estado active y la regla 22/tcp ALLOW Anywhere (con su versión IPv6), que permite únicamente el servicio SSH y bloquea el resto del tráfico entrante. Así se cumple lo que pide el taller: permitir solo los servicios necesarios.

Comandos:

- **sudo ufw enable:** “como administrador, activa el firewall”. enable significa “habilitar”.
- **sudo ufw status:** “como administrador, muestra el estado del firewall”.

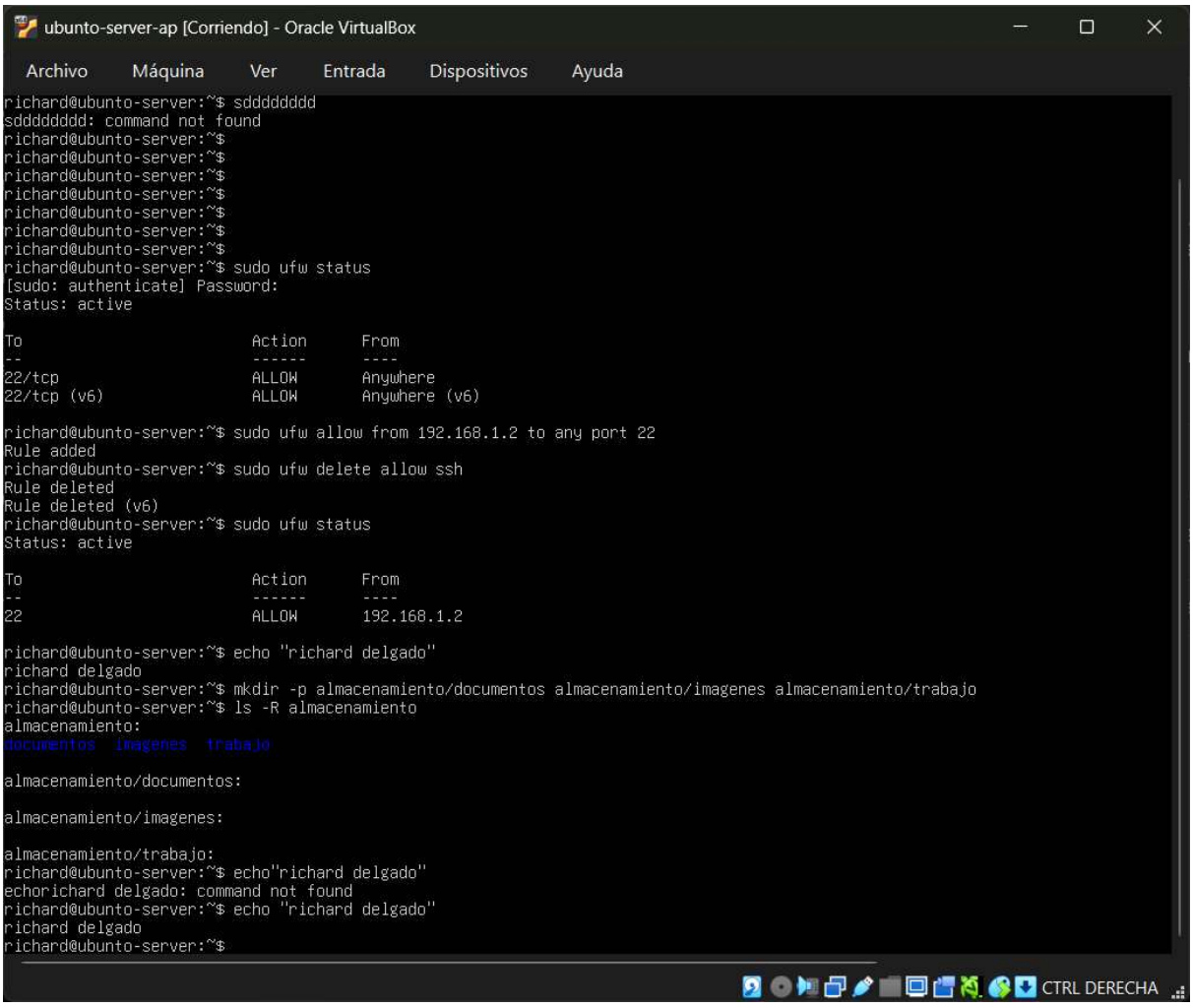


14) Restricción de acceso SSH por IP de origen

Como medida de seguridad adicional, se restringió el acceso remoto para que solo el equipo cliente pueda conectarse por SSH. En la captura se observa el estado inicial del firewall, con el puerto 22 abierto a cualquier origen (Anywhere). Primero se creó una regla que permite el tráfico al puerto 22 únicamente desde la IP del cliente, con sudo ufw allow from 192.168.1.2 to any port 22. Luego se eliminó la regla general con sudo ufw delete allow ssh, que respondió Rule deleted y Rule deleted (v6). Finalmente, sudo ufw status mostró que el puerto 22 solo acepta conexiones desde 192.168.1.2. De esta forma, cualquier otro equipo de la red queda bloqueado, lo que reduce la superficie de ataque del servidor. La captura se firmó con echo "richard delgado".

Comandos:

- **sudo ufw allow from 192.168.1.2 to any port 22:** “como administrador, en el firewall, permite desde la IP 192.168.1.2 hacia cualquier dirección del servidor, en el puerto 22”.
- **sudo ufw delete allow ssh:** “como administrador, en el firewall, borra la regla que permite ssh”.
- **sudo ufw status:** “como administrador, muestra el estado del firewall”.



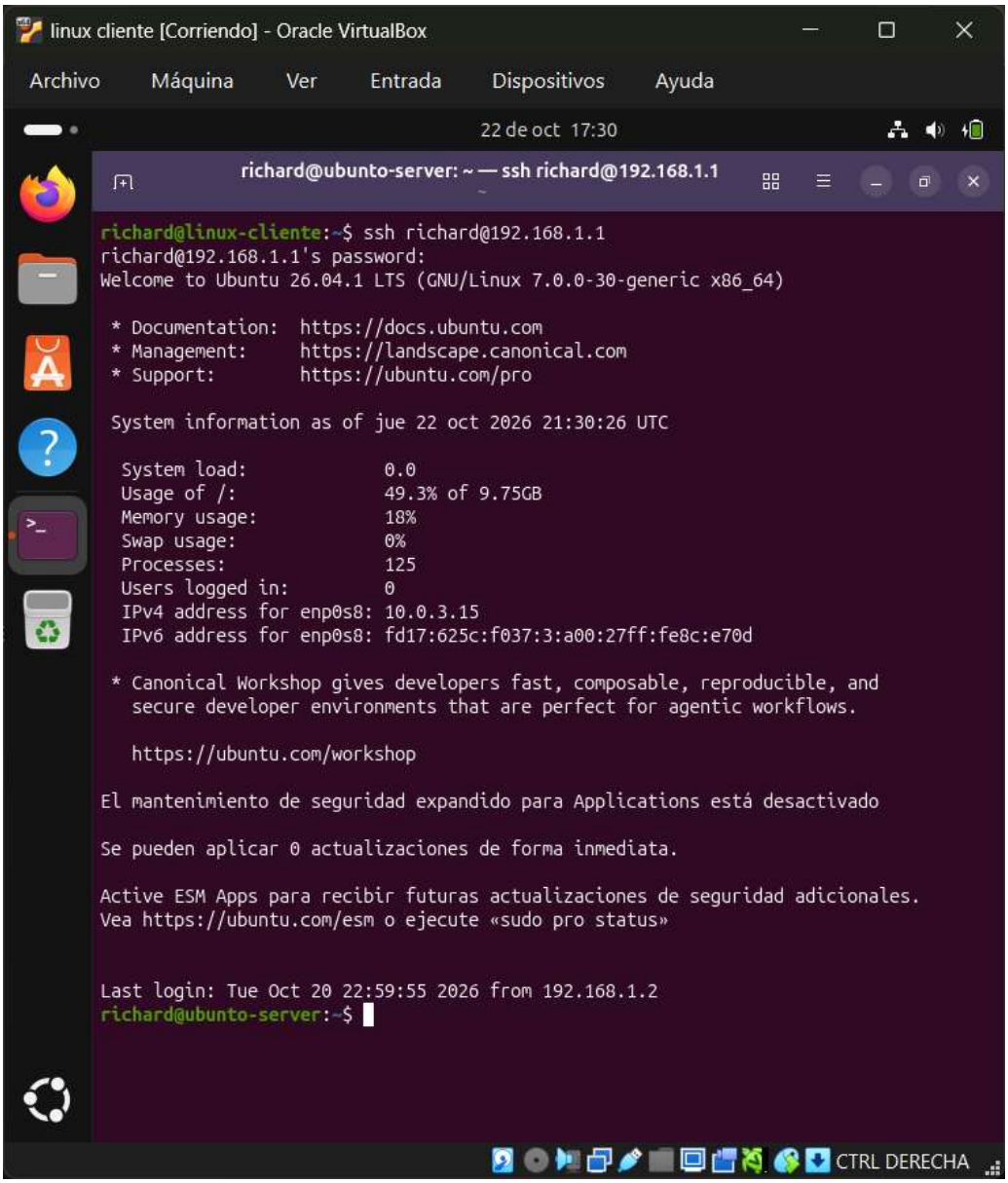
15) Creación de la estructura de directorios

Para organizar el almacenamiento en el servidor, se creó una estructura de directorios para documentos, imágenes y archivos de trabajo. Se ejecutó mkdir -p almacenamiento/documentos almacenamiento/imagenes almacenamiento/trabajo, donde la opción -p crea la carpeta principal almacenamiento junto con sus tres subcarpetas en una sola instrucción. Luego se verificó el resultado con ls -R almacenamiento, que lista el contenido de forma recursiva y mostró las tres

carpetas, aún vacías. Esta estructura recibe después los archivos transferidos desde el cliente por SCP. La captura se firmó con echo "richard delgado".

Comandos:

> **-mkdir -p almacenamiento/documentos almacenamiento/imagenes almacenamiento/trabajo**: “crea directorios, incluyendo las carpetas intermedias que falten y sin dar error si ya existen”.
> **-ls -R almacenamiento**: “lista el contenido de almacenamiento y de todas sus subcarpetas”.

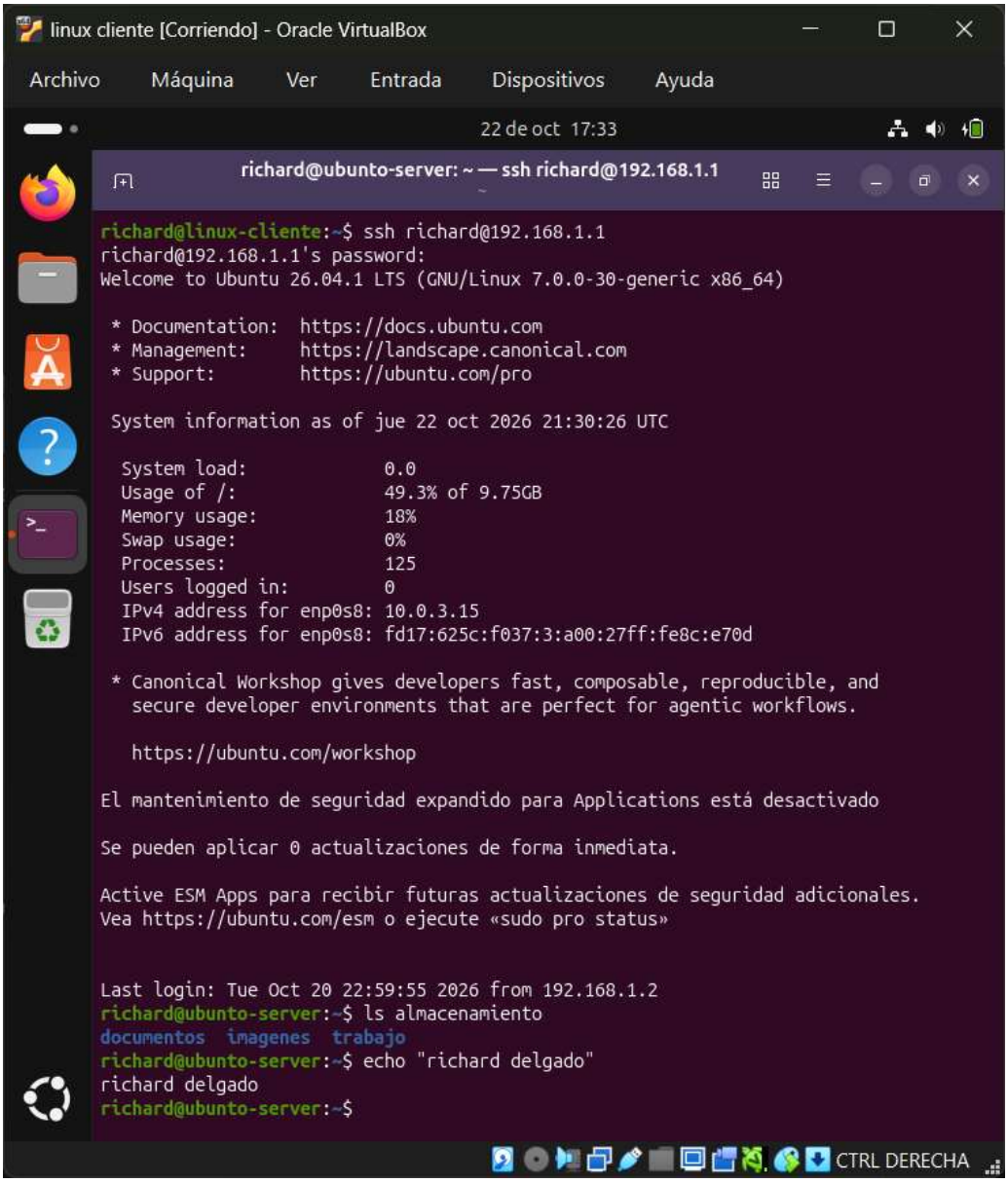


16) Verificación del acceso remoto desde el cliente

Para comprobar que la regla del firewall funciona correctamente, se realizó una conexión remota desde el equipo cliente (192.168.1.2) hacia el servidor. En la terminal del cliente se ejecutó el comando ssh richard@192.168.1.1 y, ingresada la contraseña del usuario, se obtuvo acceso al servidor, lo que se evidenció en el cambio del prompt a richard@ubuntu-server. El mensaje de bienvenida muestra el último inicio de sesión desde 192.168.1.2. Esto demuestra que el cliente autorizado por la regla sudo ufw allow from 192.168.1.2 to any port 22 puede administrar el servidor sin acceder físicamente a él.

Comandos:

- **ssh richard@192.168.1.1**: “conéctate de forma segura, como el usuario richard, al equipo con la dirección 192.168.1.1”. ssh es *Secure Shell*, lo que va antes de la arroba es el usuario y lo que va después es la dirección del servidor.



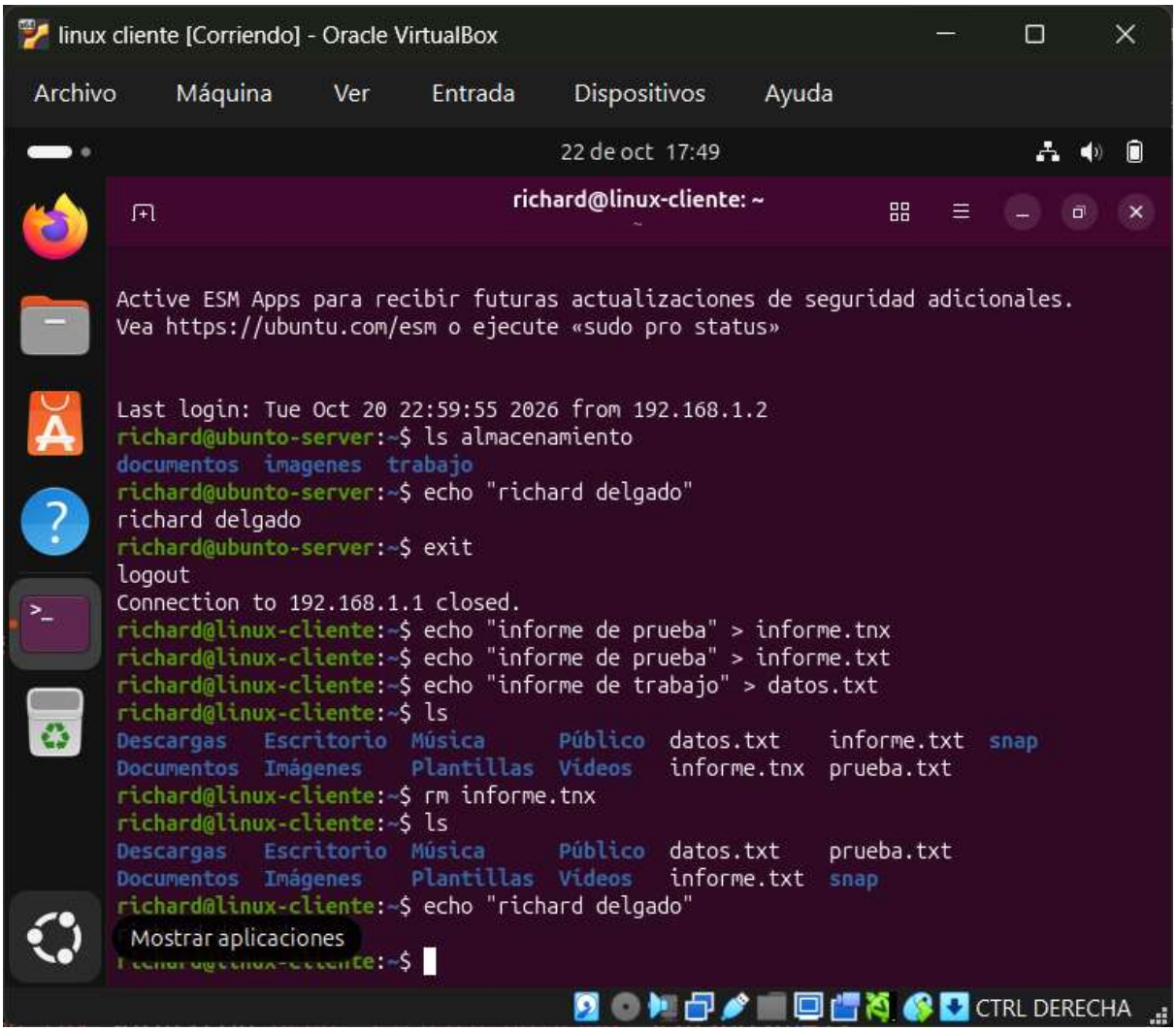
17) Verificación de las carpetas desde la sesión remota

Con la sesión SSH abierta, se ejecutó ls almacenamiento desde el cliente para comprobar que la estructura creada en el servidor es visible de forma remota. El comando mostró las tres carpetas: documentos, imagenes y trabajo. El prompt richard@ubuntu-server confirma que los comandos se ejecutaron en el servidor y no en el cliente. La captura se firmó con echo "richard delgado".

Comandos:

- **ls almacenamiento**: “lista el contenido de la carpeta almacenamiento”. A diferencia de ls -R, solo muestra el primer nivel, sin entrar en las subcarpetas.

.



18) Creación de archivos de prueba y borrado del archivo erróneo

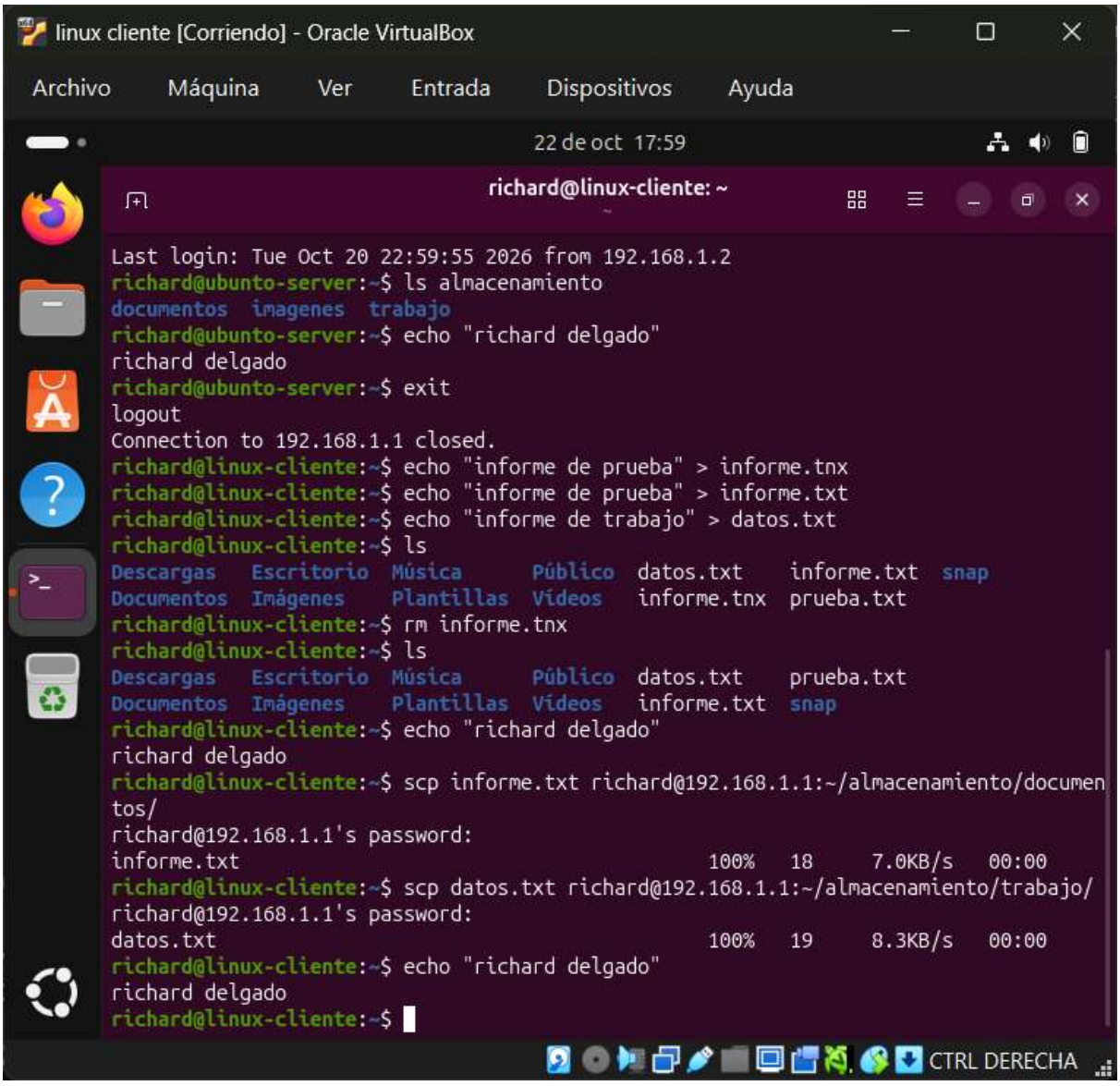
Como scp debe ejecutarse desde el equipo cliente y no desde el servidor, primero se cerró la sesión remota con exit, y el sistema respondió Connection to 192.168.1.1 closed. De vuelta en la terminal local (richard@linux-cliente), se crearon los archivos de prueba con echo "informe de prueba" > informe.txt y echo "informe de trabajo" > datos.txt. Por error de digitación, también se creó informe.tnx, que se identificó con ls junto a informe.txt y datos.txt. Ese archivo erróneo se eliminó con rm informe.tnx y se verificó con un nuevo ls, donde ya no aparece. La captura se firmó con echo "richard delgado".

Comandos:

- **exit**: “sal de la sesión actual”. En este caso cierra la conexión SSH y devuelve al cliente.
- **echo "informe de prueba" > informe.txt**: “imprime el texto entre comillas, pero en lugar de mostrarlo en pantalla, guárdalo en el archivo

informe.txt". El símbolo > redirige la salida a un archivo, y lo crea si no existe.

- **ls**: "lista los archivos y carpetas del directorio actual".
- **rm informe.tnx**: "elimina el archivo informe.tnx". rm es *remove* y borra de forma definitiva, sin pasar por la papelera.



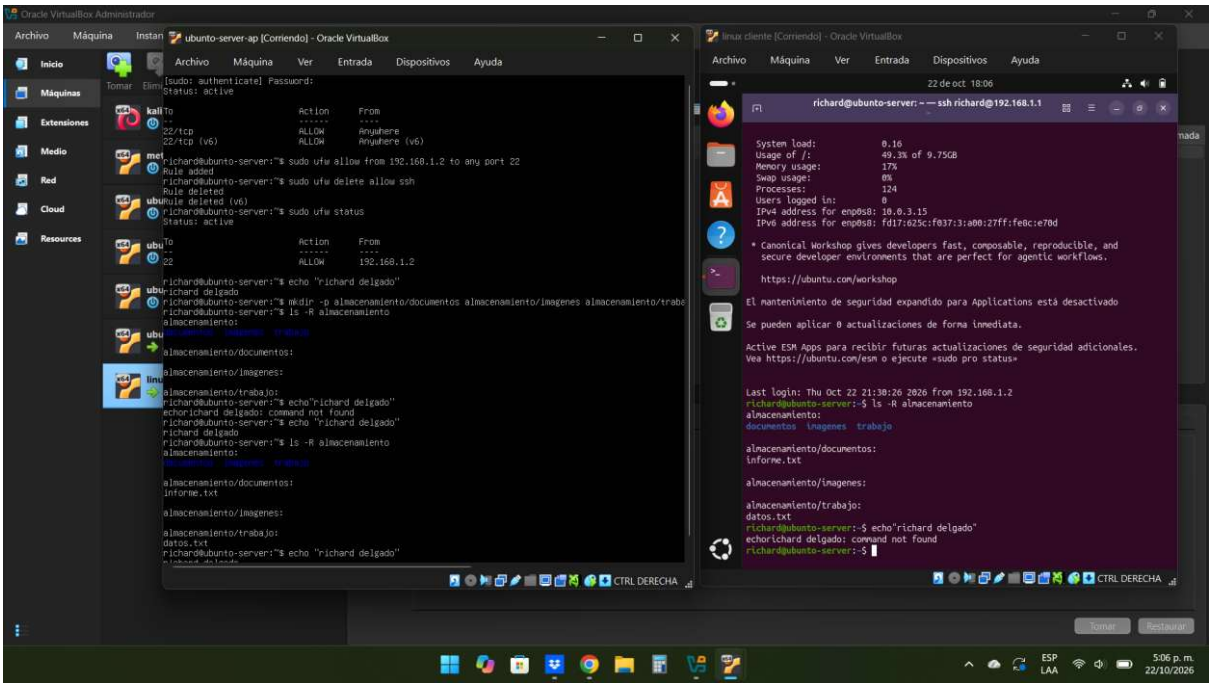
19) Transferencia de archivos con SCP

Desde la terminal del cliente (richard@linux-cliente), se enviaron los dos archivos de prueba al servidor con scp, cada uno hacia la carpeta que le corresponde. Primero se ejecutó scp informe.txt richard@192.168.1.1:~/almacenamiento/documentos/ y, tras ingresar la contraseña de richard, el sistema mostró informe.txt 100% 18, es decir, el archivo se transfirió completo (18 bytes). Luego se ejecutó scp datos.txt richard@192.168.1.1:~/almacenamiento/trabajo/, que mostró datos.txt 100% 19, con

el mismo resultado. La transferencia fue posible porque la regla de UFW permite el acceso desde la IP del cliente, 192.168.1.2. La captura se firmó con echo "richard delgado"

Comandos:

- **scp informe.txt richard@192.168.1.1:~/almacenamiento/documentos/**: “copia de forma segura el archivo informe.txt al equipo 192.168.1.1, como el usuario richard, dentro de la carpeta almacenamiento/documentos de su directorio personal”. scp es *Secure Copy* y usa SSH para transferir. Lo que va antes de los dos puntos es el origen y lo que va después es el destino. El símbolo ~ representa la carpeta personal del usuario.
- **scp datos.txt richard@192.168.1.1: ~/almacenamiento/trabajo/**: “copia de forma segura el archivo datos.txt al servidor, dentro de almacenamiento/trabajo”.

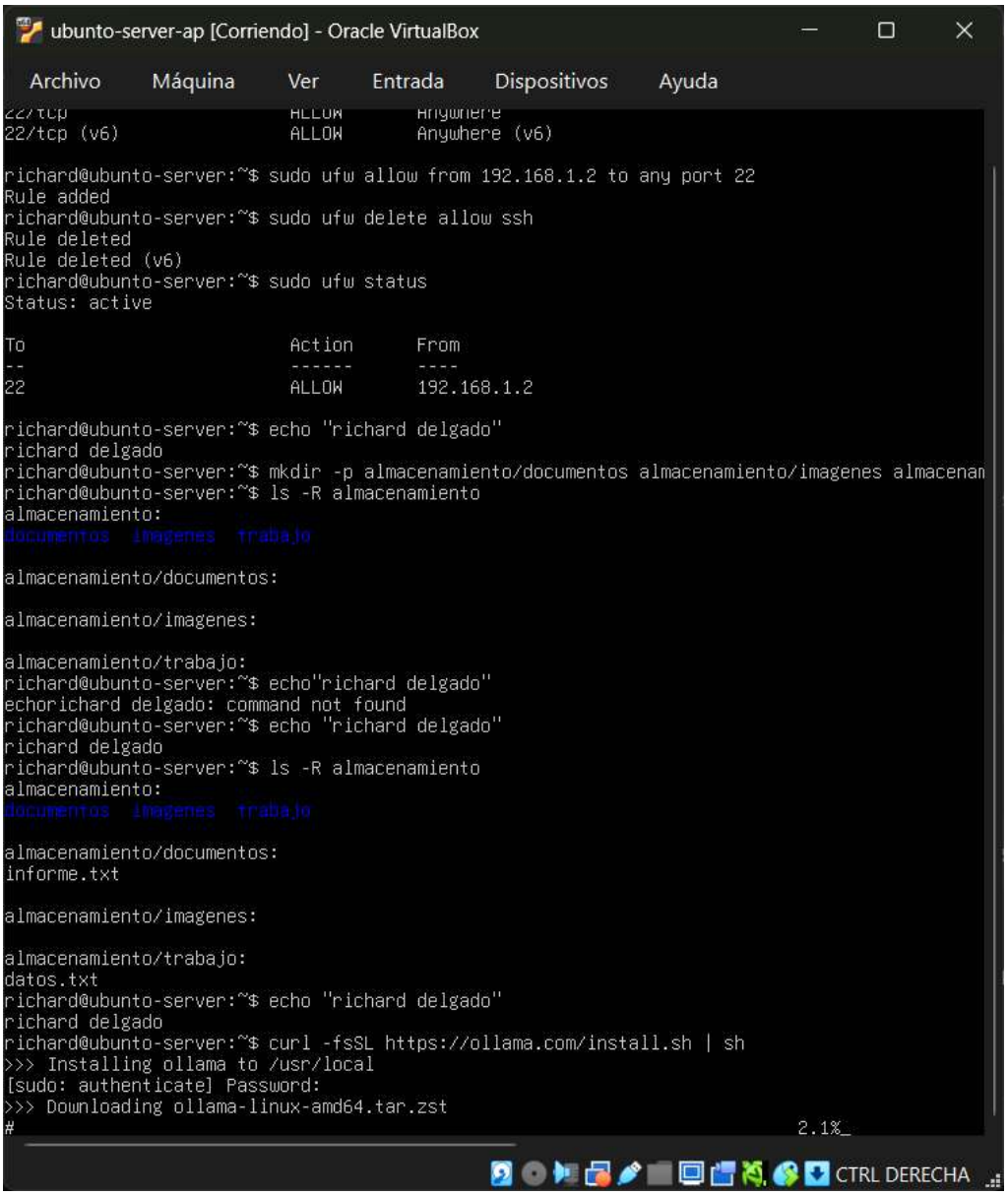


20) Verificación de la llegada de los archivos

Para confirmar que la transferencia con scp fue exitosa, se mostraron las dos máquinas virtuales al mismo tiempo en VirtualBox. A la derecha, desde el cliente, se abrió una nueva sesión SSH con ssh richard@192.168.1.1 y se ejecutó ls -R almacenamiento. El resultado muestra informe.txt dentro de documentos y datos.txt dentro de trabajo, mientras que imagenes sigue vacía. A la izquierda, la consola del servidor (ubuntu-server) muestra el mismo resultado con el mismo comando. Esto confirma que los archivos enviados desde el cliente llegaron a la carpeta que les correspondía.

Comandos:

- **ssh richard@192.168.1.1**: “conéctate de forma segura, como el usuario richard, al equipo 192.168.1.1”.
- **ls -R almacenamiento**: “lista el contenido de almacenamiento y, de forma recursiva, el de todas sus subcarpetas”. -R significa *recursive*.



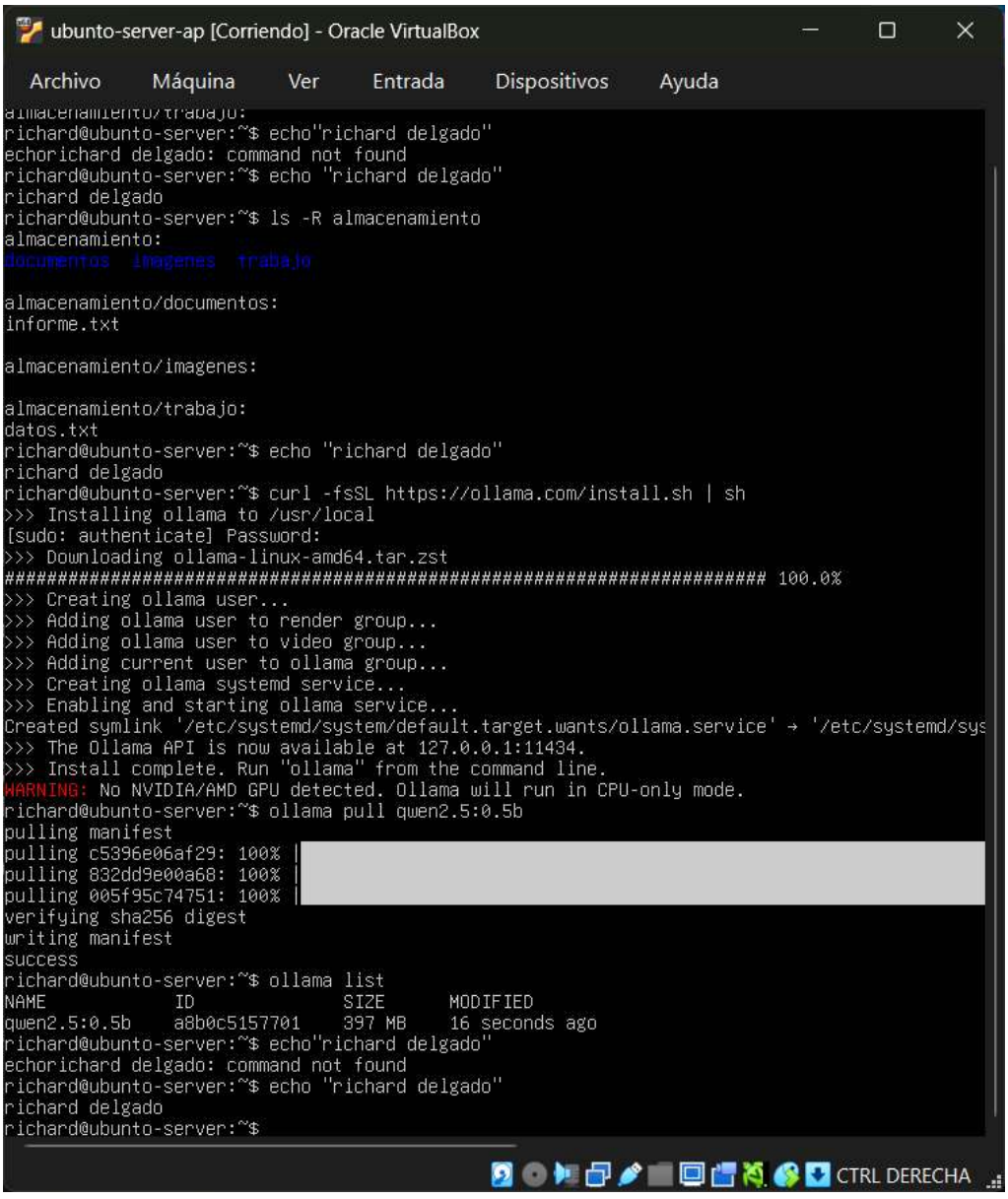
21) Instalación de Ollama en el servidor

Se instaló Ollama en el servidor con el comando curl -fsSL https://ollama.com/install.sh | sh, que descarga el programa por el adaptador NAT y lo instala en /usr/local. Antes de instalar, ls -R almacenamiento confirmó que informe.txt y datos.txt estaban en sus carpetas. Tras pedir la contraseña de

administrador, comenzó la descarga del paquete ollama-linux-amd64.tar.zst, que en la captura va en 2,1 %.

Comandos:

- **curl -fsSL https://ollama.com/install.sh | sh**: “descarga el script de instalación desde esa dirección y ejecútalo de inmediato”. curl descarga contenido de internet y la tubería | pasa lo descargado a sh, que lo ejecuta.
- **ls -R almacenamiento**: “lista el contenido de almacenamiento y de todas sus subcarpetas”.



22) Finalización de la instalación, aviso de GPU y descarga del modelo

Al terminar la instalación, el instalador creó el usuario ollama, lo agregó a los grupos render y video, y registró un servicio de systemd que se inicia automáticamente. Confirmó que la API de Ollama quedó disponible de forma local en 127.0.0.1:11434, es decir, accesible solo desde el propio servidor.

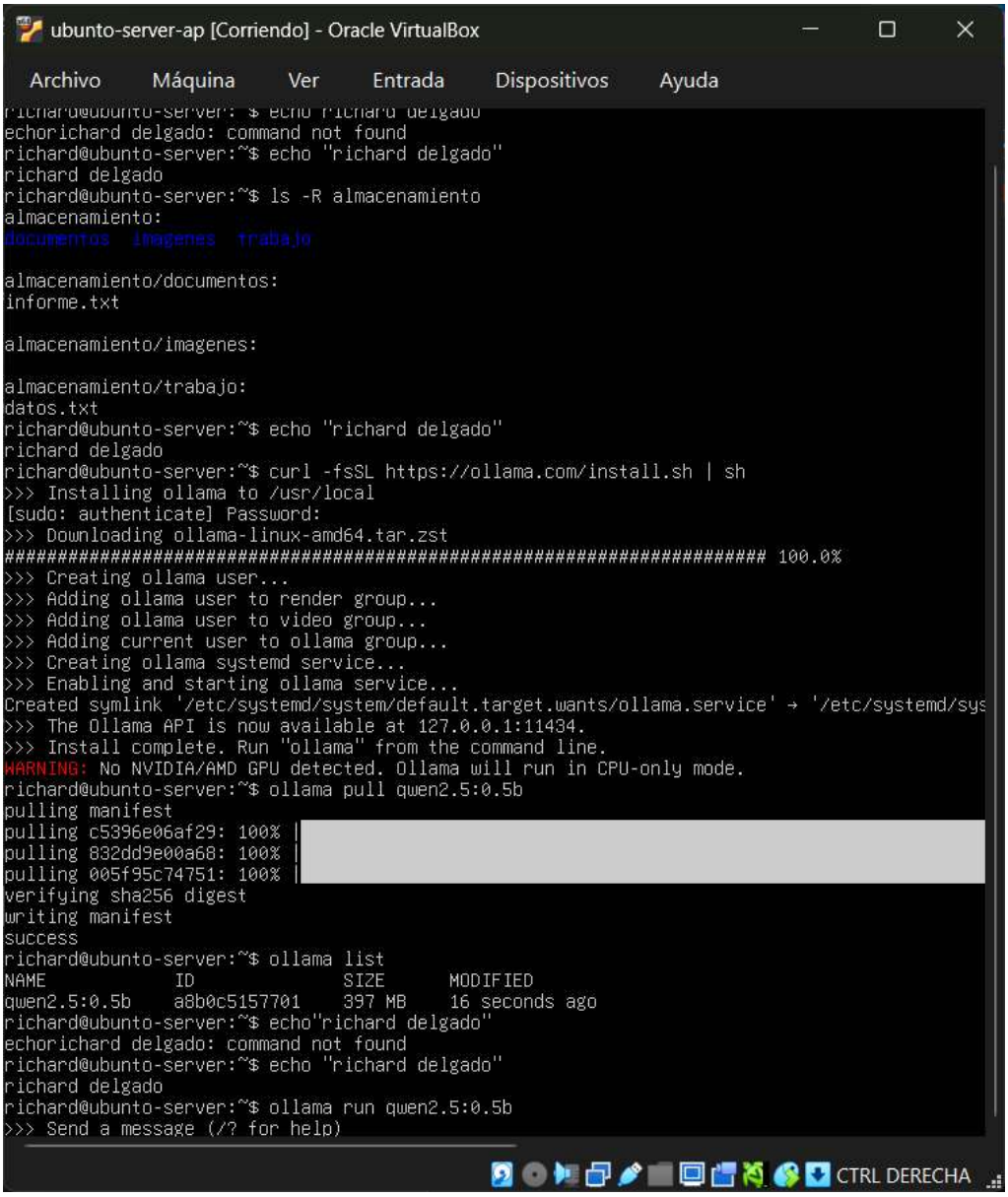
Apareció un aviso en rojo: “No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode”. No es un error: la máquina virtual no tiene tarjeta gráfica, así que Ollama usa solo el procesador. Es suficiente para un modelo liviano, aunque las respuestas serán más lentas.

Luego se descargó el modelo con ollama pull qwen2.5:0.5b, que terminó con success, y se comprobó con ollama list, que muestra qwen2.5:0.5b con un tamaño de 397 MB.

En la captura también aparece un error de digitación: se escribió echo"richard delgado" sin espacio y el sistema respondió command not found. Se corrigió en la línea siguiente, agregando el espacio.

Comandos:

- **ollama pull qwen2.5:0.5b**: “descarga el modelo qwen2.5:0.5b y guárdalo en el equipo”. pull significa “traer”, y lo que va después de los dos puntos indica la versión del modelo (0,5 mil millones de parámetros).
- **ollama list**: “muestra los modelos instalados”, con su nombre, identificador y tamaño.



23) Inicio del modelo con ollama run

Con el modelo ya descargado, se ejecutó ollama run qwen2.5:0.5b para iniciar una sesión interactiva con la IA local. El sistema mostró el indicador >>> Send a message (/? for help), que significa que el modelo está cargado y espera la primera pregunta. Todo se procesa en el propio servidor, sin conexión a servicios externos.

Comandos:

- **ollama run qwen2.5:0.5b**: “ejecuta el modelo qwen2.5:0.5b y abre una conversación con él en la terminal”. run significa “ejecutar”. Si el modelo no estuviera descargado, Ollama lo descargaría primero.
- **>>> Send a message (/? for help)**: no es un comando, es el indicador del modelo. Significa “envía un mensaje” y, si se escribe /?, muestra la ayuda.



24) Prueba de la IA local

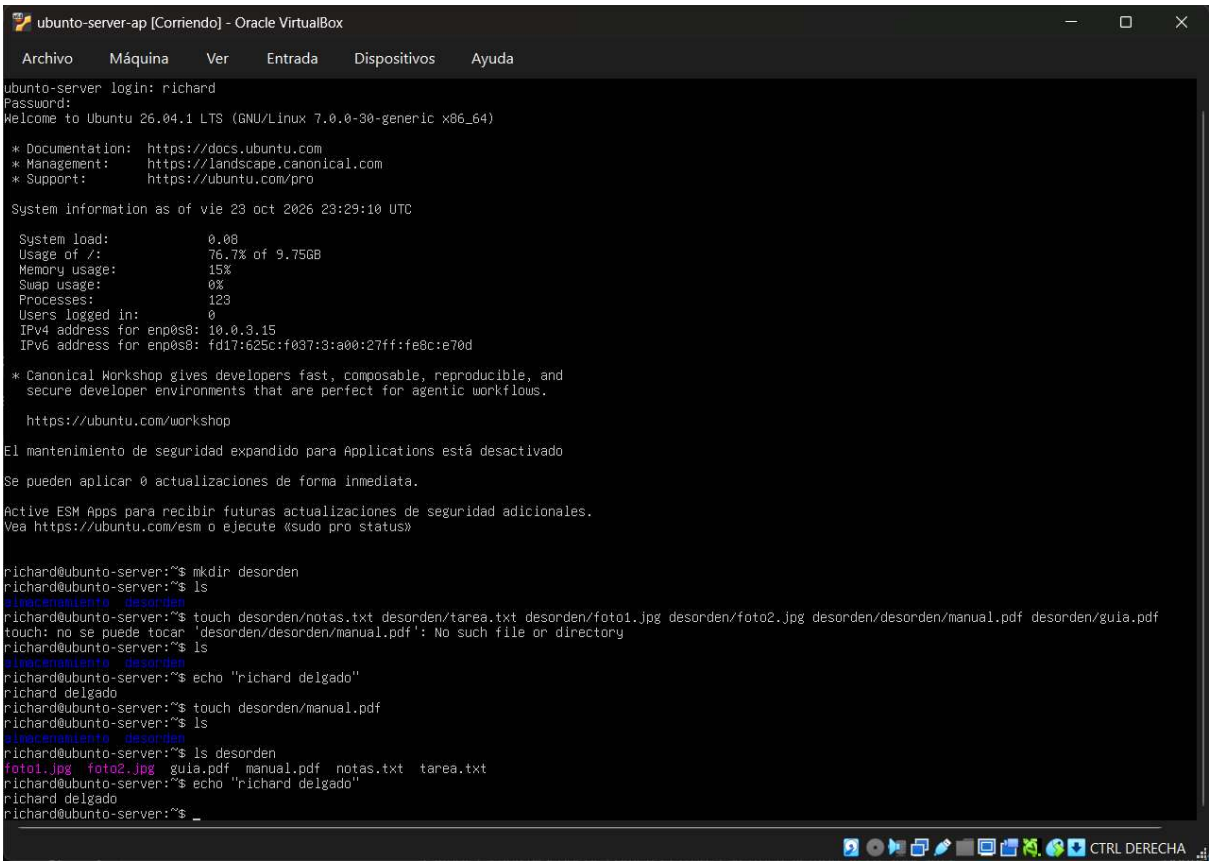
Con el modelo iniciado, se le hicieron preguntas desde la terminal para comprobar que responde. Primero se escribió un saludo, hola ¿cómo me te va?, y el modelo contestó en español. Luego se preguntó ¿para qué sirve el comando chmod en linux?. La respuesta fue en parte correcta, porque indicó que chmod cambia los permisos de archivos y directorios. Sin embargo, describió mal las opciones: llamó “permitidos”, “reales” o “generalizados” a permisos que en realidad son lectura (r), escritura (w) y ejecución (x). Esto confirma que las respuestas de un modelo tan pequeño pueden contener errores y deben revisarse antes de usarse.

Mientras generaba la respuesta aparecieron mensajes del sistema (watchdog: BUG: soft lockup y rcu_preempt self-detected stall). Indican que el procesador de la máquina virtual quedó saturado por el esfuerzo del modelo, lo que explica la lentitud y que el texto se viera desordenado. No detuvo la sesión.

Después se escribieron graccias, fvsd y bye como mensajes normales, y el modelo los respondió como texto. La sesión se cerró con /bye, que devolvió al indicador richard@ubuntu-server.

Comandos:

- **ollama run qwen2.5:0.5b**: “ejecuta el modelo y abre una conversación con él en la terminal”.
- **/bye**: “cierra la sesión con el modelo y vuelve a la terminal”. Se escribe con la barra porque es un comando de Ollama. Sin la barra, bye se envía al modelo como un mensaje más.



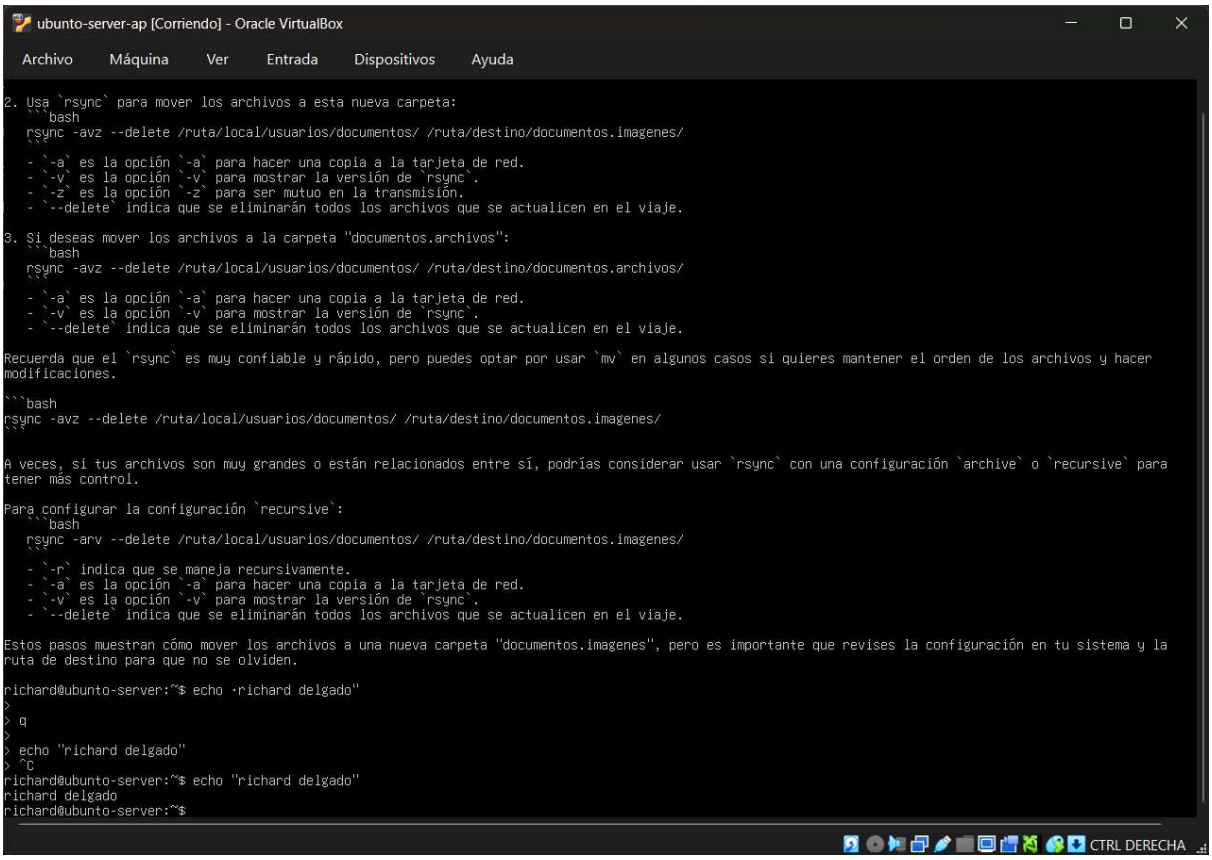
25) Creación de la carpeta desorden con archivos mezclados

Para simular un servidor con información desordenada, se creó la carpeta desorden con mkdir desorden, y se comprobó con ls, que mostró almacenamiento y desorden. Dentro de ella se crearon seis archivos vacíos de distintos tipos con touch: notas.txt, tarea.txt, foto1.jpg, foto2.jpg, manual.pdf y guia.pdf.

En la captura aparece un error de digitación: la ruta de manual.pdf se escribió como desorden/desorden/manual.pdf, y el sistema respondió No such file or directory, porque esa subcarpeta no existe. Los otros cinco archivos sí se crearon. El error se corrigió ejecutando touch desorden/manual.pdf. Finalmente, ls desorden mostró los seis archivos.

Comandos:

- **mkdir desorden**: “crea un directorio llamado desorden”.
- **touch desorden/notas.txt desorden/tarea.txt ...**: “crea archivos vacíos con esos nombres dentro de desorden”.
- **ls**: “lista los archivos y carpetas del directorio actual”.
- **ls desorden**: “lista el contenido de la carpeta desorden”.



26) Consulta a la IA sobre cómo organizar los archivos

Para obtener una estrategia de clasificación, se consultó a la IA local desde la consola del servidor con ollama run qwen2.5:0.5b "pregunta". En la captura se ve el final de su respuesta, que propone usar rsync -avz --delete /ruta/local/usuarios/documentos/ /ruta/destino/documentos.imagenes/ y una variante con -arv.

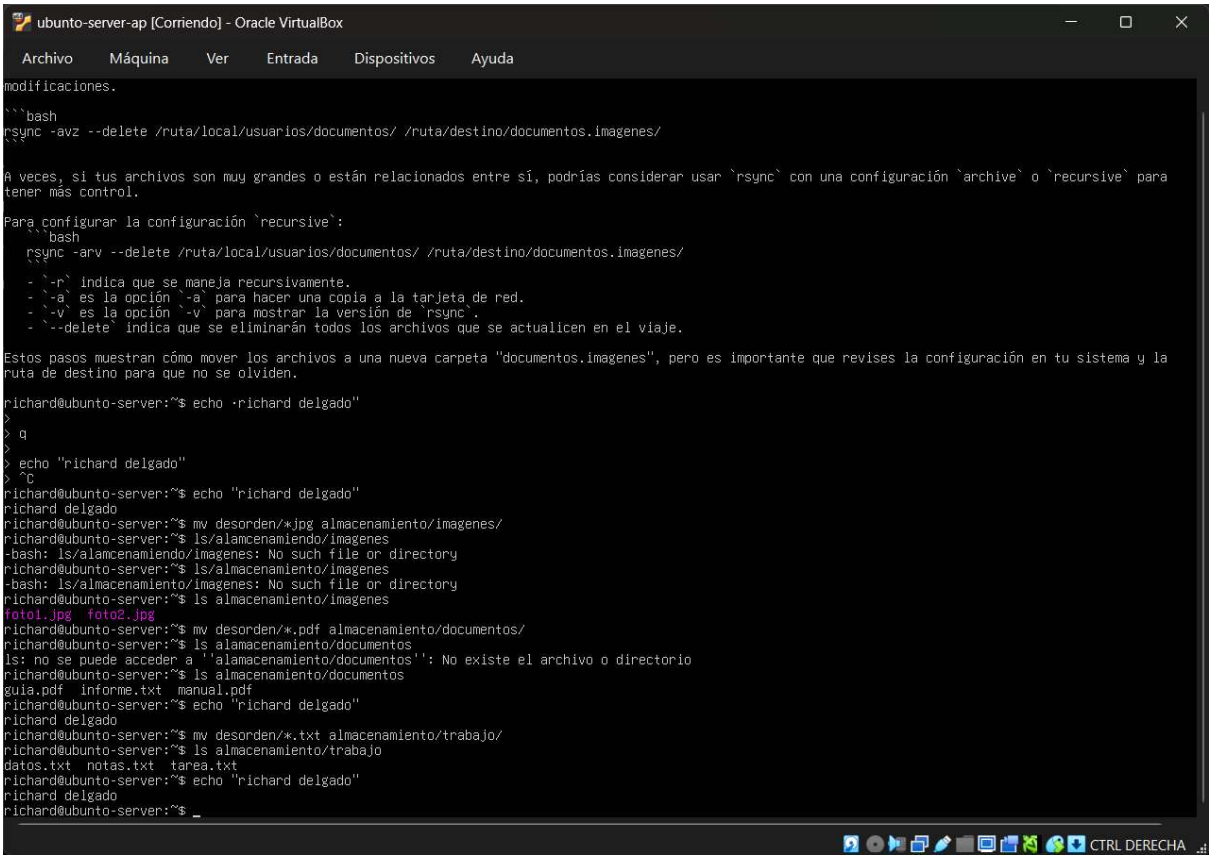
La propuesta se analizó antes de aplicarla y se descartó por estos motivos:

- **Inventa carpetas que no existen**, como documentos.imagenes y documentos.archivos, y rutas genéricas (/ruta/local/usuarios/...) que no corresponden al servidor.
- **Recomienda --delete**, opción de rsync que borra en el destino lo que no existe en el origen. Es peligrosa aquí: si se sincronizara desorden/ hacia almacenamiento/documentos/, se borraría informe.txt, que ya se había enviado con scp.
- **Explica mal las opciones**: por ejemplo, dice que -v muestra la versión de rsync, cuando en realidad muestra más detalle de lo que se hace (*verbose*), y que -a copia “a la tarjeta de red”.
- **Solo acierta en lo básico**: crear carpetas y clasificar por tipo de archivo (txt, jpg, pdf).

Por ello no se aplicó tal cual. Se conservó únicamente la idea de clasificar por extensión y se ejecutó con mv, un comando más simple y seguro.

Comandos:

- **ollama run qwen2.5:0.5b "pregunta"**: “ejecuta el modelo y envíale esa pregunta; responde en la terminal y termina”.
- **rsync -avz --delete origen/ destino/**: “sincroniza el destino para que sea idéntico al origen”. -a conserva la estructura y los permisos, -v muestra el detalle, -z comprime durante la transferencia y --delete borra en el destino lo que no está en el origen. Este comando solo se analizó, no se ejecutó.



27) Clasificación de los archivos por extensión con mv

Con base en la idea de clasificar por tipo, se movieron los archivos de desorden a su carpeta con mv y el comodín *, que representa “todos los archivos con esa terminación”. Primero se movieron las imágenes con mv desorden/*.jpg almacenamiento/imagenes/, y ls almacenamiento/imagenes mostró foto1.jpg y foto2.jpg. Luego se movieron los documentos PDF con mv desorden/*.pdf almacenamiento/documentos/, y ls almacenamiento/documentos mostró guia.pdf, informe.txt y manual.pdf. Por último, se movieron los archivos de texto con mv desorden/*.txt almacenamiento/trabajo/, y ls almacenamiento/trabajo mostró datos.txt, notas.txt y tarea.txt.

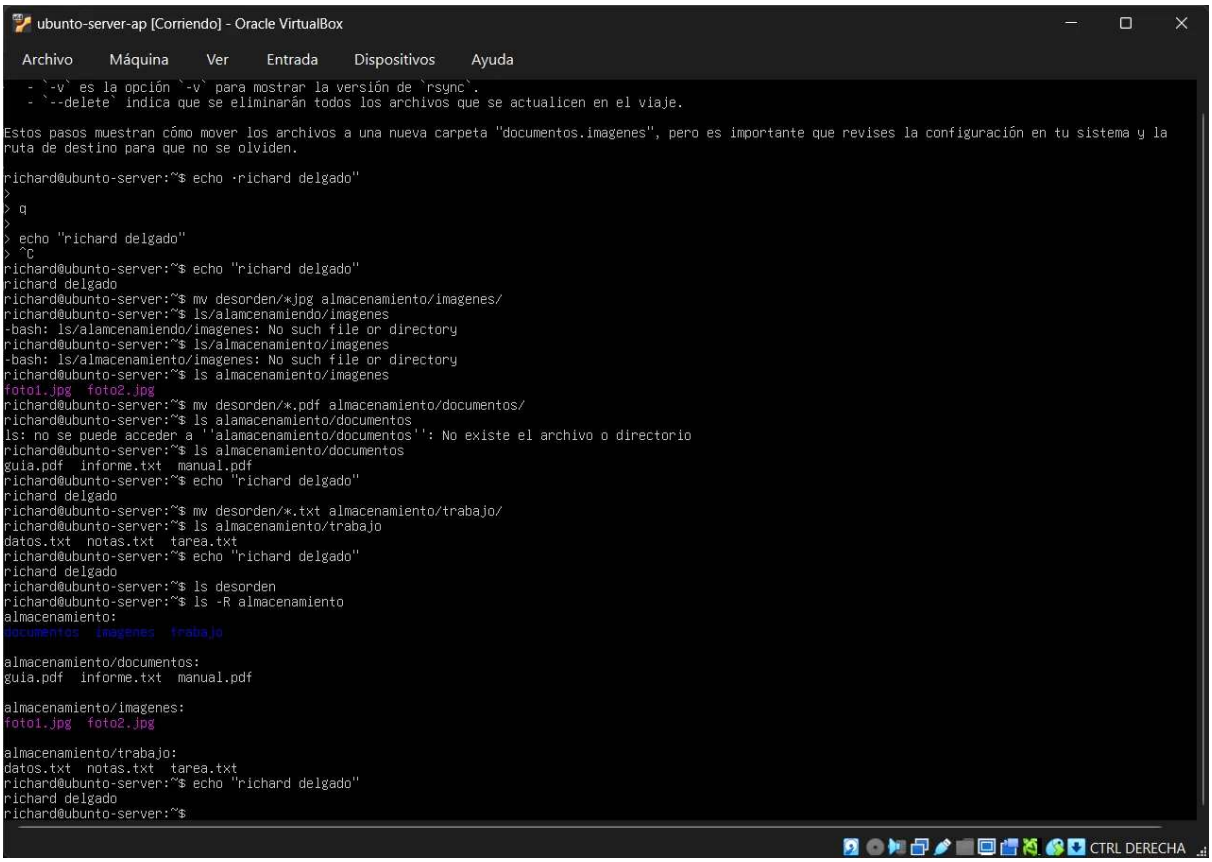
En la captura aparecen errores de digitación al verificar. Dos veces se escribió ls/almacenamiento/imagenes sin espacio después de ls (una de ellas también con el nombre mal escrito), y el sistema respondió No such file or directory. Otra vez se escribió ls alamacenamiento/documentos, con el nombre mal escrito, y respondió No existe el archivo o directorio. En los tres casos se corrigió repitiendo el comando bien escrito, y los archivos ya estaban en su lugar.

Comandos:

- **mv desorden/*.jpg almacenamiento/imagenes/**: “mueve todos los archivos que terminan en .jpg de la carpeta desorden a la carpeta

almacenamiento/imagenes". mv es *move* (mover) y * reemplaza cualquier nombre.

- **mv desorden/*.pdf almacenamiento/documentos/**: "mueve todos los archivos .pdf de desorden a almacenamiento/documentos".
- **mv desorden/*.txt almacenamiento/trabajo/**: "mueve todos los archivos .txt de desorden a almacenamiento/trabajo".
- **ls almacenamiento/imagenes**, **ls almacenamiento/documentos** y **ls almacenamiento/trabajo**: "lista el contenido de esa carpeta", para comprobar que los archivos llegaron.

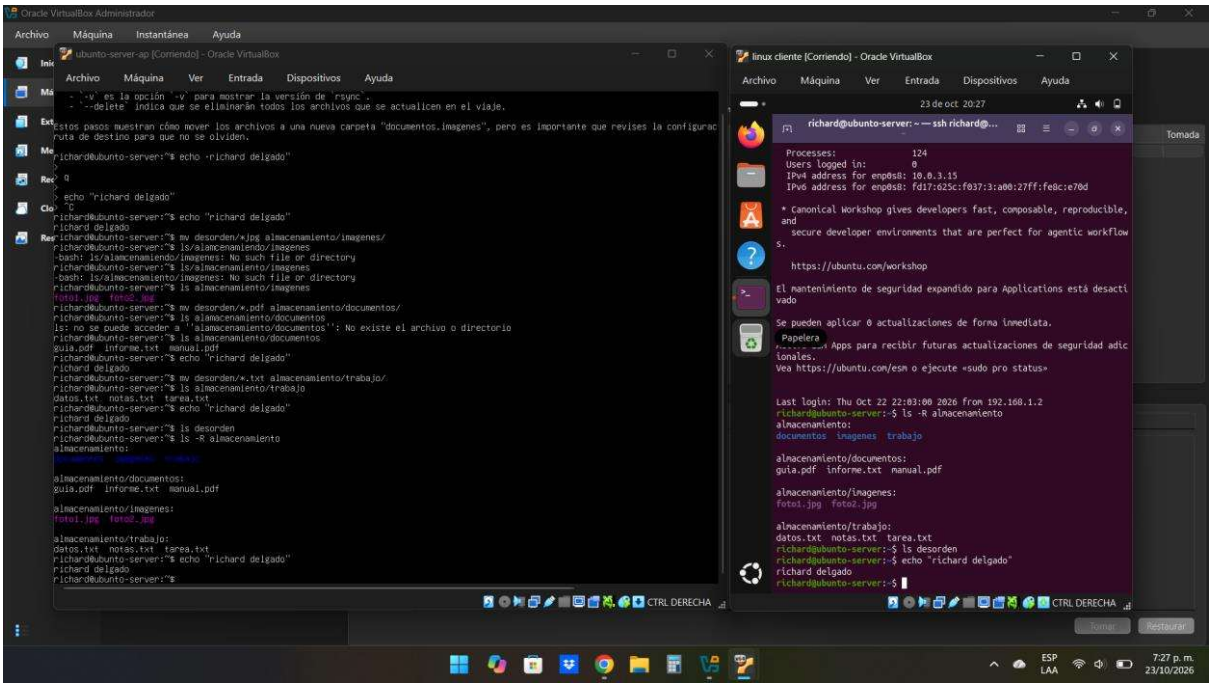


28) Verificación de la organización en el servidor

Terminados los movimientos, se comprobó el resultado con dos comandos. Primero ls desorden, que no devolvió ningún archivo, lo que confirma que la carpeta de origen quedó vacía. Luego ls -R almacenamiento, que mostró cada archivo en su carpeta: guia.pdf, informe.txt y manual.pdf en documentos; foto1.jpg y foto2.jpg en imagenes; y datos.txt, notas.txt y tarea.txt en trabajo. Así se comprueba que la clasificación por extensión se aplicó correctamente y que no se perdió ningún archivo.

Comandos:

- **ls desorden**: “lista el contenido de la carpeta desorden”. Si no muestra nada, la carpeta está vacía.
- **ls -R almacenamiento**: “lista el contenido de almacenamiento y, de forma recursiva, el de todas sus subcarpetas”.



29) Verificación final desde el cliente

Para validar la organización desde el equipo cliente, se estableció una conexión remota al servidor con ssh richard@192.168.1.1, permitida por la regla de UFW que solo acepta la IP 192.168.1.2. Dentro de la sesión se ejecutó ls -R almacenamiento, que mostró cada archivo en su carpeta: guia.pdf, informe.txt y manual.pdf en documentos; foto1.jpg y foto2.jpg en imagenes; y datos.txt, notas.txt y tarea.txt en trabajo. Además, ls desorden no devolvió ningún archivo, lo que confirma que la carpeta de origen quedó vacía. Así se comprueba, desde el cliente, que el servidor quedó ordenado, como pide el taller.

Comandos:

- **ssh richard@192.168.1.1**: “conéctate de forma segura, como el usuario richard, al equipo 192.168.1.1”.
- **ls -R almacenamiento**: “lista el contenido de almacenamiento y, de forma recursiva, el de todas sus subcarpetas”.
- **ls desorden**: “lista el contenido de la carpeta desorden”. Si no muestra nada, la carpeta está vacía.